WORK IN PROGRESS

# AI-Assisted Cryptographic Control Plane

Project Evidence Pack

Product requirements, architecture, detailed design, assurance, roadmap and reproducible evidence

Version 0.9.0

Milestones 1-9 reference release

11 October 2026

# Contents

This combined pack records the current project position. Implemented, partial, planned and out-of-scope capabilities are deliberately separated.

**AI-Assisted Cryptographic Control Plane . . . 8**

Project progress summary . . . 8

Project overview . . . 8

Work completed . . . 8

Milestone 1 - Certificate monitoring foundation . . . 8

Milestone 2 - Evidence intake and replay protection . . . 8

Milestone 3 - Cryptographic inventory and PQC assessment . . . 8

Milestone 4 - Controlled discovery laboratory . . . 9

Milestone 5 - PQC migration management . . . 9

Milestone 6 - Evidence reconciliation and offline exchange . . . 9

Milestone 7 - Authorised pilot reference gate . . . 10

Milestone 8 - Production control-plane reference foundation . . . 10

Milestone 9 - Proposal-only integration boundary . . . 10

Current evidence . . . 10

Current limitations . . . 10

Next external validation gates . . . 11

Overall position . . . 11

**Product Requirements Document . . . 12**

1. Document control . . . 12

2. Product statement . . . 12

3. Problem . . . 12

4. Standards context . . . 12

5. Users and stakeholders . . . 13

6. Goals . . . 13

7. Non-goals . . . 13

8. Product scope by capability . . . 14

9. Functional requirements . . . 14

Discovery and evidence . . . 14

Inventory and assessment . . . 15

Migration management . . . 15

Detection and assistance . . . 15

10. Non-functional requirements . . . 16

11. Success measures . . . 16

12. Acceptance criteria for the current reference release . . . 17

13. Dependencies and assumptions . . . 17

14. Open product decisions . . . 18

**High-Level Design . . . 19**

1. Architectural intent . . . 19

2. Context . . . 19

3. Logical components . . . 20

4. Trust zones . . . 20

5. Principal data flows . . . 20

5.1 Observation to inventory . . . 20

5.2 Observation to alert . . . 21

5.3 Inventory to migration work (target) . . . 21

6. Data domains . . . 21

7. Deployment view . . . 22

Current reference deployment . . . 22

Target production deployment . . . 22

8. Availability and recovery concept . . . 22

9. Security architecture . . . 22

10. Key architecture decisions . . . 23

11. Architecture limitations . . . 23

12. Version 0.9 reference-release components . . . 23

**Low-Level Design . . . 24**

1. Scope . . . 24

2. Package structure . . . 24

3. Contract catalogue . . . 24

4. Certificate event processing . . . 25

4.1 Validation sequence . . . 25

4.2 Evidence-claim policy . . . 25

5. Monitoring rule design . . . 25

CERT-001: unknown DNS namespace . . . 25
6. CT fixture collection . . . 26
6.1 Bounds . . . 26
6.2 Replay algorithm . . . 26
7. Persistence model . . . 26
8. Inventory projection . . . 27
9. PQC assessment algorithm . . . 27
10. X.509 discovery laboratory . . . 27
10.1 Synthetic factory . . . 27
10.2 File parser . . . 27
10.3 Loopback TLS probe . . . 28
11. Command interfaces . . . 28
12. Implemented Milestone 5 data types . . . 28
Enterprise catalogue record . . . 28
Migration work item . . . 29
Decision record . . . 29
Exception record . . . 29
Verification record . . . 29
13. Migration state machine . . . 29
14. Error-handling rules . . . 29
15. Observability requirements for production . . . 30
16. Implemented Milestones 6 to 9 reference types . . . 30
**Requirements Traceability Matrix . . . 31**
1. Purpose . . . 31
2. Implemented and partial requirements . . . 32
3. Planned requirements and required proof . . . 33
4. Security invariant traceability . . . 34
5. Traceability maintenance rule . . . 34
**Verification and Validation Plan . . . 35**
1. Objective . . . 35
2. Test levels . . . 35
3. Current automated acceptance suite . . . 35
4. Reproduction procedure . . . 36

5. Golden-scenario validation . . . 36
6. Production-readiness validation gates . . . 36
Gate A: authenticated pilot intake . . . 36
Gate B: inventory quality . . . 36
Gate C: migration workflow . . . 37
Gate D: production service . . . 37
Gate E: optional execution . . . 37
7. Test data strategy . . . 37
8. Milestone 5 verification result . . . 37
9. Milestones 6 to 9 reference verification result . . . 38
**Security, Risk and Assurance Plan . . . 39**
1. Assurance position . . . 39
2. Assets requiring protection . . . 39
3. Primary threat scenarios . . . 40
4. Control objectives . . . 40
Evidence trust . . . 40
Decision integrity . . . 40
AI governance . . . 40
Execution safety . . . 40
5. Risk register . . . 41
6. Assurance artefacts by gate . . . 41
7. Compliance mapping approach . . . 41
8. Review cadence . . . 41
**Delivery Roadmap and Governance . . . 43**
1. Completed foundation . . . 43
2. Reference implementation status and external gates . . . 43
Milestone 6 - Evidence reconciliation and offline bundle: locally implemented . . . 43
Milestone 7 - Authorised read-only pilot: reference gate implemented . . . 43
Milestone 8 - Production control-plane foundation: local reference implemented . . . 43
Milestone 9 - Optional controlled integrations: proposal boundary implemented . . . 44
3. Governance roles . . . 44
4. Decision gates . . . 44

5. Delivery controls . . . 44
6. Backlog priority method . . . 45
7. Change control . . . 45
8. Immediate external-evidence backlog . . . 45

**Project Evidence and Demonstration Guide . . . 47**
1. Claim being demonstrated . . . 47
2. Evidence baseline . . . 47
3. Environment setup . . . 47
4. Automated proof . . . 47
5. Demonstration A - deterministic certificate alert . . . 47
6. Demonstration B - replay-safe evidence intake . . . 48
7. Demonstration C - PQC inventory assessment . . . 48
8. Demonstration D - real X.509 parsing . . . 48
9. Demonstration E - security boundaries . . . 48
10. Reviewer checklist . . . 48
11. Evidence manifest template . . . 49
12. Milestone 5 migration-management demonstration . . . 49
13. Milestones 6 to 9 reference-release demonstration . . . 50

**ADR-001: Contract-first deterministic certificate monitoring slice . . . 51**
Context . . . 51
Decision . . . 51
Consequences . . . 51
Guardrails for later milestones . . . 51

**ADR-002: Trustworthy intake contracts and replay-safe fixture collection . . . 53**
Context . . . 53
Decision . . . 53
Consequences . . . 53
Rejected alternatives . . . 54

**ADR-003: Evidence-backed cryptographic inventory and PQC exposure . . . 55**
Context . . . 55
Decision . . . 55
Safety semantics . . . 55
Consequences . . . 55
Rejected alternatives . . . 56

**ADR-004: Controlled discovery lab without customer data . . . 57**
Context . . . 57

Decision . . . 57
Consequences . . . 57
Rejected alternatives . . . 58

**ADR-005: Deterministic migration governance without execution authority . . . 59**
Context . . . 59
Decision . . . 59
Consequences . . . 59
Rejected alternatives . . . 60

**ADR-006: Safe reference foundations for Milestones 6 to 9 . . . 61**
Context . . . 61
Decision . . . 61
Consequences . . . 61
Rejected alternatives . . . 62

**Threat model: Milestones 1 to 9 reference release . . . 63**
Assets . . . 63
Trust boundaries . . . 63
Threats and current mitigations . . . 64
Security invariants . . . 65
Milestones 6 to 9 threat-model delta . . . 66

New assets . . . 66

New threats and controls . . . 66

Residual risk . . . 66
Out of scope . . . 66

**SOC playbook: CERT-001 - certificate for an unknown DNS namespace . . . 67**
Purpose . . . 67
Initial triage . . . 67
Escalation . . . 67
Containment and recovery . . . 67
Closure evidence . . . 67

# AI-Assisted Cryptographic Control Plane

## Project progress summary

**Project status:** Work in progress

**Current version:** 0.9.0

**Current milestone:** Milestone 6 implemented; Milestones 7-9 reference foundations implemented with external gates open

**Last updated:** 11 October 2026

## Project overview

This project is developing an **evidence-backed cryptographic discovery and post-quantum cryptography (PQC) migration control plane**.

Its purpose is to help organisations discover where cryptography is being used, record the supporting evidence, identify cryptographic components that may be affected by quantum computing, and manage future migration work in a controlled and auditable way.

Certificate monitoring is the first working discovery capability. The intended product is broader and will eventually include cryptographic inventory, dependencies, ownership, migration planning, approvals, exceptions, and verification.

## Work completed

### Milestone 1 - Certificate monitoring foundation

- Created a versioned certificate-event format.
- Added strict validation and DNS-name normalisation.
- Added deterministic detection of certificates containing unapproved DNS

namespaces.

- Added risk scoring, evidence-based explanations, alert output, and a SOC

playbook.

### Milestone 2 - Evidence intake and replay protection

- Added a controlled Certificate Transparency fixture collector.
- Added evidence provenance and SHA-256 content digests.
- Added SQLite storage for observations, outcomes, and checkpoints.
- Added duplicate detection, restart support, and fail-closed provenance

conflict handling.

### Milestone 3 - Cryptographic inventory and PQC assessment

- Added cryptographic inventory records linked to source evidence.
- Added deterministic identification of quantum-vulnerable RSA and

elliptic-curve certificate evidence.

- Added support for representing ML-KEM, ML-DSA, SLH-DSA, hybrid, and unknown

algorithm evidence.

- Added explicit handling of missing ownership, business criticality, data classification, and protection-lifetime information.
- Prevented PQC algorithm names from being treated as proof that a deployment is quantum-safe.

## Milestone 4 - Controlled discovery laboratory

- Added generation of real RSA and elliptic-curve X.509 test certificates.
- Added PEM and DER certificate parsing.
- Added discovery of certificate fingerprints, identities, validity periods, public-key algorithms, key sizes, curves, signatures, key usages, and extended key usages.
- Added a bounded loopback TLS test that records the TLS version and cipher suite without scanning external systems.
- Added evidence-maturity levels that distinguish simulated, generated, sanitised, public-observation, authenticated-production, and independently verified evidence.
- Added controls preventing laboratory evidence from being presented as authenticated production evidence.

## Milestone 5 - PQC migration management

- Added a fictional six-service enterprise catalogue with assets, owners, suppliers, business context, cryptographic uses, and dependencies.
- Added deterministic migration priorities, waves, blockers, and function-specific target compatibility.
- Added separately versioned work-item, decision, exception, verification, and coverage-report contracts.
- Added controls for dependency cycles, ownership conflicts, evidence escalation, stale decisions, self-approval, expired exceptions, and incomplete or non-independent verification.
- Added separate discovery-coverage and verified-migration measures.
- Kept all implementation activity outside the control plane; it remains a planning, governance, and verification reference implementation.

## Milestone 6 - Evidence reconciliation and offline exchange

- Added deterministic multi-observation correlation that retains all record IDs and exposes contradictory algorithm states.
- Added CBOM-style and dependency-graph exports.
- Added field-level sanitisation that preserves cryptographic properties without upgrading evidence maturity.

- Added Ed25519 manifest signing and import validation. Embedded self-signed

keys establish local integrity only; signer trust is separate.

### Milestone 7 - Authorised pilot reference gate

- Added time-bounded, independently approved, read-only scope contracts.
- Added fail-closed expiry, self-authorisation, source-authentication, and asset-

scope checks.

- Added denominator-based coverage reporting. The included run is simulated and

does not claim that a real pilot has taken place.

### Milestone 8 - Production control-plane reference foundation

- Added tenant-scoped workflow persistence, optimistic version enforcement,

role checks, separation of duties, audit records, and transactional outbox records in local SQLite.

- Production identity, availability, recovery, SLO, privacy, and independent

assurance gates remain open.

### Milestone 9 - Proposal-only integration boundary

- Added validated proposals for ticketing, CMDB, PKI, KMS/HSM, and deployment-

evidence integration families.

- All proposals have `execution_authority: none`; cryptographic and deployment

mutations are rejected.

## Current evidence

- Nine roadmap milestones are represented in the reference design; Milestones

7-9 retain explicit external acceptance gates.

- The repository contains versioned JSON schemas, architecture decisions, a

threat model, examples, command-line demonstrations, and automated tests.

- The current automated suite contains **75 tests**.
- The current full run passes on Python 3.13. The loopback TLS test passes when

run outside the restricted test sandbox. A fresh 3.12 run remains pending a test environment containing the pinned project dependencies.

- Milestone 4 is recorded in commit `be71182` and the supporting project

documentation is recorded in later local commits.

## Current limitations

The project is a local reference implementation and remains under development. It does not currently:

- connect to customer or production data sources;
- claim complete discovery of an organisation's cryptographic estate;
- provide authenticated production collectors;
- discover every form of cryptography used in applications, code, KMS, HSM,

private PKI, cloud services, or supplier products;

- operate as a production multi-user or multi-tenant service;
- execute certificate, key, trust-store, or production migration changes;
- prove that an organisation is quantum-safe;
- carry independent certification, audit, or penetration-test assurance.

These limitations are intentional at the current stage. The project is proving the data contracts, evidence boundaries, deterministic assessment rules, safety controls, and migration-management design before production integrations or change authority are introduced.

## Next external validation gates

The next work is evidence, not another feature milestone: obtain written pilot authority and a bounded read-only source; validate an external signer trust registry; review privacy and retention; select a production identity/tenant architecture; demonstrate backup, recovery, SLOs, and independent security testing; then review each proposed adapter before granting any authority.

## Overall position

The project has progressed beyond a concept or presentation. It has executable code, real cryptographic test artifacts, machine-readable contracts, persistent evidence handling, documented security boundaries, and repeatable automated tests.

It should presently be described as a **working reference implementation under active development**, not as a finished production product. The completed work now demonstrates both evidence-backed discovery and controlled PQC migration management using explicitly simulated enterprise data. It also demonstrates the safe architecture of later roadmap stages without claiming their external gates have been passed.

# Product Requirements Document

## 1. Document control

| Field | Value |
|---|---|
| Product | AI-assisted Cryptographic Control Plane |
| Product version | 0.9.0 reference implementation |
| Document status | Baseline for review |
| Date | 2026-10-11 |
| Product owner | To be assigned |
| Technical owner | Repository owner |
| Implementation status | Milestones 1-6 implemented locally; Milestones 7-9 safe reference foundations with external gates open |

## 2. Product statement

The product is an evidence-backed cryptographic discovery and PQC migration control plane. It collects bounded observations, normalises them into portable contracts, creates a cryptographic inventory, classifies exposure using deterministic policy, and will govern migration work through accountable human decisions and verifiable completion evidence.

Certificate monitoring is the first sensor and reference vertical slice. It is not the final product boundary.

## 3. Problem

PQC migration is not only an algorithm-replacement exercise. Organisations need to know:

- where cryptography is used;
- which function each primitive performs;
- which assets, services, owners, data, suppliers, and relying parties depend

on it;

- how reliable each observation is and what it does not prove;
- which systems require earlier action because protected data must remain

confidential for a long period;

- whether products and protocols support an approved target state;
- how changes are approved, verified, rolled back, and evidenced.

Without those controls, an organisation can produce an inventory yet still be unable to prioritise or prove a migration.

## 4. Standards context

The design uses the functional distinction established by the final NIST PQC standards:

- FIPS 203 specifies ML-KEM for key establishment:

<https://csrc.nist.gov/pubs/fips/203/final>

- FIPS 204 specifies ML-DSA for digital signatures:

<https://csrc.nist.gov/pubs/fips/204/final>

- FIPS 205 specifies SLH-DSA for digital signatures:

<https://csrc.nist.gov/pubs/fips/205/final>

NIST's PQC programme states that organisations should identify vulnerable algorithm use and begin migration planning: <https://csrc.nist.gov/projects/post-quantum-cryptography>

NIST IR 8547 is transition guidance and remains identified as an initial public draft at this document date. Its proposed dates must therefore be treated as planning inputs, not silently converted into universal legal deadlines: <https://csrc.nist.gov/pubs/ir/8547/ipd>

## 5. Users and stakeholders

| Persona | Need |
|---|---|
| Cryptography or PKI lead | Evidence of algorithms, certificates, protocols, trust relationships, and migration state |
| Security architect | Dependency and target-state views with explicit assumptions |
| Asset or service owner | Assigned, understandable migration work and acceptance criteria |
| Risk and compliance reviewer | Traceable decisions, exceptions, approvals, evidence, and coverage gaps |
| SOC analyst | Actionable certificate anomalies with source evidence and playbooks |
| Platform or application engineer | Implementation-specific compatibility requirements and test results |
| Executive sponsor | Exposure, coverage, blockers, forecast, and residual-risk reporting |
| Auditor or independent reviewer | Immutable provenance, change history, control evidence, and reproducibility |

## 6. Goals

1 Build an evidence-backed, continuously reconcilable cryptographic inventory.

2 Preserve the distinction between observed fact, inference, recommendation, approval, execution, and verified outcome.

1 Prioritise PQC work using cryptographic exposure, data lifetime, business criticality, dependency, supplier, and readiness evidence.

1 Manage migration waves, blockers, exceptions, approvals, rollback criteria, and verification evidence.

1 Make coverage and unknowns visible rather than treating missing evidence as safety.

1 Keep AI advisory and bounded; deterministic policy and accountable humans retain authority for material decisions.

## 7. Non-goals

The current product is not:

- a certificate authority, KMS, HSM, secret manager, or trust store;
- an autonomous certificate or key lifecycle executor;
- a vulnerability scanner authorised to probe arbitrary networks;
- proof that any real organisation is quantum-safe;
- a replacement for protocol-specific interoperability testing;
- a compliance certification engine;
- a guarantee that every cryptographic use can be discovered automatically.

## 8. Product scope by capability

| Capability | Current state | Target state |
|---|---|---|
| Canonical certificate observations | Implemented | Multiple authenticated collectors |
| Schema and domain validation | Implemented | Contract compatibility and producer conformance programme |
| Certificate anomaly detection | Implemented | Governed rule catalogue and production SIEM delivery |
| Replay-safe fixture intake | Implemented | Distributed, authenticated intake with transactional delivery |
| Cryptographic inventory | Partially implemented | Multi-source CBOM with reconciliation and lineage |
| PQC exposure assessment | Partially implemented | Policy-versioned portfolio prioritisation |
| Real X.509 parsing | Implemented for controlled files | Sandboxed production parsing and chain verification |
| TLS evidence | Implemented for loopback lab | Authorised scoped collectors with protocol evidence |
| Dependency graph | Partially implemented with fictional catalogue | Reconciled service, protocol, supplier, and relying-party graph |
| Migration workflow | Partially implemented as local deterministic workflow | Durable waves, tasks, approvals, exceptions, rollback, verification |
| AI assistance | Deterministic explanation only | Evidence-grounded recommendation with evaluation and guardrails |
| Execution | Out of scope now | Optional isolated executor after governance gates |

## 9. Functional requirements

### Discovery and evidence

- **FR-001:** Accept only versioned, validated observation contracts.
- **FR-002:** Preserve source identity, collection time, payload digest, verification state, evidence level, and cursor where applicable.
- **FR-003:** Reject replay conflicts where a source identity is reused with different content.
- **FR-004:** Parse X.509 evidence to derive, not merely accept, algorithm and certificate properties.
- **FR-005:** Keep certificate, transport, code, key-store, and configuration observations distinct until correlation is supported by evidence.
- **FR-006:** Attach explicit scope and coverage limitations to inventory data.
- **FR-007:** Support future collectors for CT, TLS, cloud certificate managers, private PKI, KMS/HSM, code/configuration, and supplier evidence.

## Inventory and assessment

- **FR-010:** Create stable inventory records linked to their observations.
- **FR-011:** Represent cryptographic function separately from algorithm name.
- **FR-012:** Classify RSA, elliptic-curve, and finite-field public-key evidence as quantum-vulnerable for PQC planning.
- **FR-013:** Treat ML-KEM, ML-DSA, SLH-DSA, and hybrid labels as candidates requiring deployment and interoperability verification.
- **FR-014:** Preserve unknown values and prevent them from becoming safe defaults.
- **FR-015:** Block final migration priority when required business context is missing.
- **FR-016:** Correlate assets, services, owners, data lifetime, suppliers, dependencies, and cryptographic uses in the target product.

## Migration management

- **FR-020:** Define an approved target state per cryptographic function and protocol, not one universal replacement algorithm.
- **FR-021:** Group work into migration waves based on exposure, urgency, dependency order, readiness, and change risk.
- **FR-022:** Record owner, accountable approver, planned dates, blockers, dependencies, rollback criteria, and verification criteria.
- **FR-023:** Manage time-bounded exceptions with rationale, compensating controls, expiry, and named approval.
- **FR-024:** Require pre-change approval and post-change verification evidence for material migrations.
- **FR-025:** Reconcile post-change discovery against the approved target and detect regression.
- **FR-026:** Report coverage separately from migration completion.

## Detection and assistance

- **FR-030:** Evaluate security-critical detections and priority gates using deterministic, versioned policy.
- **FR-031:** Produce explanations linked to evidence and policy versions.
- **FR-032:** Prevent an AI component from silently changing evidence, severity, approval, or execution state.
- **FR-033:** Route alerts and workflow events to controlled downstream systems

with delivery evidence in the target product.

## 10. Non-functional requirements

- **NFR-001 Security:** Fail closed for invalid contracts and unsupported evidence claims.
- **NFR-002 Auditability:** Every material conclusion must identify its input evidence, rule or model version, and decision actor.
- **NFR-003 Privacy:** Minimise, classify, redact, encrypt, and retain sensitive infrastructure metadata according to policy.
- **NFR-004 Availability:** The future service must define recovery objectives, backpressure, retry, and degradation modes before production use.
- **NFR-005 Isolation:** Collection, analysis, approval, and execution must have separate trust and privilege boundaries.
- **NFR-006 Portability:** External contracts use versioned JSON Schema; core decisions must not depend on a single vendor's data model.
- **NFR-007 Explainability:** Assessment outcomes must be reproducible without access to an LLM.
- **NFR-008 Testability:** Each mandatory requirement needs an objective acceptance test or an explicitly recorded evidence gap.
- **NFR-009 Scalability:** The target architecture must partition by tenant and source and support asynchronous processing; SQLite is not the scale design.
- **NFR-010 Accessibility:** Human workflows and reports must be usable without requiring cryptography-specialist knowledge for every field.

## 11. Success measures

The measures must never combine coverage and migration into one misleading percentage.

| Measure | Definition |
|---|---|
| Discovery coverage | In-scope assets with at least one current observation divided by known in-scope assets |
| Context completeness | Inventory records with required owner, service, criticality, classification, and lifetime fields |
| Evidence maturity | Distribution of records by evidence level and verification state |
| Exposure backlog | In-scope quantum-vulnerable uses not yet assigned to an approved wave |
| Dependency readiness | Migration items whose blocking dependencies have an accepted target and tested path |
| Migration verified | Items with approved change, required tests, post-change evidence, and no unresolved regression |
| Exception health | Active, expired, and overdue exceptions by owner and risk |
| Detection quality | Validated true/false positive rates by rule and evidence source |

No production target is set until an authorised pilot establishes the asset denominator, data quality, operating model, and acceptable error rates.

## 12. Acceptance criteria for the current reference release

1. All published example payloads conform to their schemas.
2. Invalid or unexpected event fields fail closed.
3. DNS matching is normalised and restricted to label boundaries.
4. Alert identifiers and PQC assessments are deterministic.
5. Fixture replay is idempotent and changed-content conflicts fail closed.
6. Source evidence, provenance, inventory, and assessment are linked.
7. Unknown algorithm evidence is never labelled quantum-resistant.
8. Generated X.509 certificates are parsed for RSA/ECDSA properties.
9. The lab TLS probe refuses non-loopback targets.
10. Transport evidence does not invent negotiated key establishment.
11. Fixtures and generated evidence cannot claim production assurance.
12. The complete automated test suite passes on the supported Python versions.
13. Catalogue identities, ownership, scope, and dependency graph validate.
14. Missing evidence, stale evidence, unsupported targets, and blocked hard dependencies remain explicit blockers.
1. Migration decisions bind to an exact work-item version and prevent requester self-approval.
1. Exceptions are separate, time bounded, and include compensating controls.
2. `verified` requires independent pre-change/post-change evidence and every approved criterion to pass.
1. Discovery coverage and migration completion are reported separately.
2. Reconciliation retains every source record and labels contradictory states.
3. Sanitisation and bundle signing do not increase evidence maturity.
4. Offline import detects altered names, sizes, hashes, manifests, or signatures.
5. Pilot measurement requires current, independent, read-only authorisation and rejects records outside its asset denominator.
1. Tenant workflow access, approval role, separation of duties, and expected object version are enforced before state change.
1. Audit and outbox records are committed with the workflow transition.
2. Integration records remain approval-required proposals with no execution authority, and mutating cryptographic operations are rejected.
1. The reference release reports production gates as open.

## 13. Dependencies and assumptions

- Python 3.11 or later.
- Pinned `jsonschema[format-nongpl]` and `cryptography` dependencies.
- The current repository operates locally and has no production credentials.
- Future production use assumes legal authorisation, collector identity,

authoritative scope, secure hosting, operational ownership, and incident response.

## 14. Open product decisions

1. First authorised pilot source and accountable data owner.
2. Tenant and identity model.
3. Authoritative asset and ownership sources.
4. Workflow system of record and separation-of-duties model.
5. Policy ownership and exception authority.
6. Evidence retention, residency, and redaction rules.
7. Deployment model and service-level objectives.
8. Whether execution will remain external or use an isolated adapter.

# High-Level Design

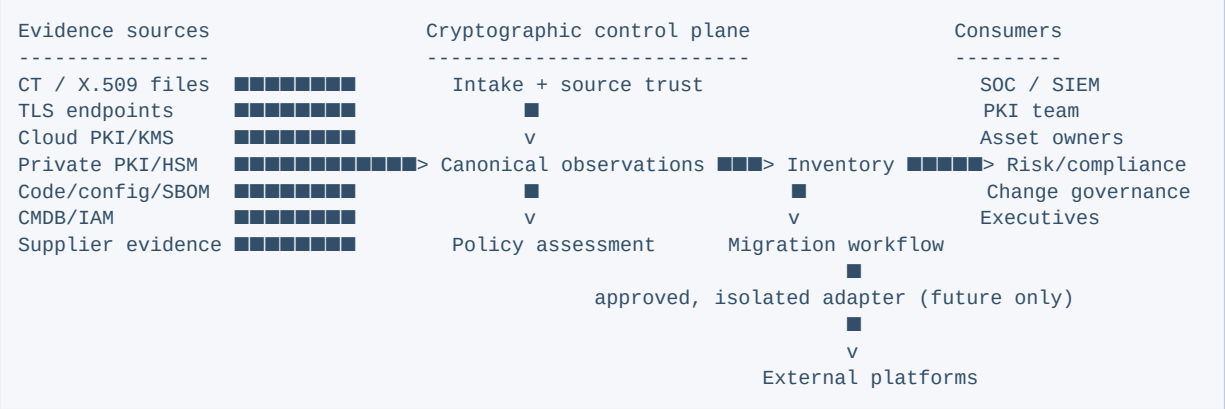
## 1. Architectural intent

The system is a control plane for evidence, decisions, and migration governance. It is not itself a cryptographic data plane. Applications, PKI platforms, TLS stacks, KMS/HSM products, and cloud services continue to perform cryptographic operations.

The architecture follows five rules:

1. Evidence precedes conclusions.
2. Missing evidence remains visible.
3. Deterministic policy owns security-critical decisions.
4. Approval and execution are separate trust boundaries.
5. Migration completion requires post-change verification.

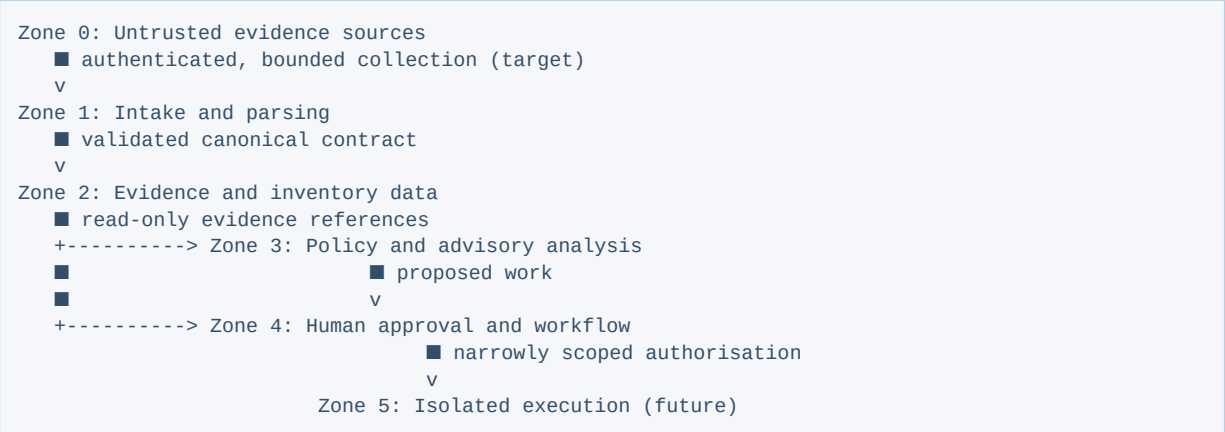
## 2. Context



# 3. Logical components

| Component | Responsibility | Current status |
|---|---|---|
| Collector adapters | Acquire bounded evidence from an authorised source | Fixture, file, and loopback lab implemented |
| Intake gateway | Authenticate source, enforce limits, validate contract, attach receipt metadata | Contract validation implemented; authentication planned |
| Evidence ledger | Preserve raw observation, digest, provenance, outcome, and lineage | SQLite reference implementation |
| Normalisation service | Convert source-specific data into canonical observations | Certificate path implemented |
| Inventory service | Produce cryptographic-use records with coverage limits | Certificate slice implemented |
| Correlation and graph service | Link assets, services, identities, dependencies, suppliers, and relying parties | Local conflict-preserving reconciliation and fictional dependency graph implemented; authoritative sources open |
| Policy and assessment service | Classify exposure, missing context, priority gates, and policy findings | Deterministic certificate/PQC slice implemented |
| Detection service | Produce operational alerts and playbook links | CERT-001 implemented |
| AI advisory service | Summarise evidence, suggest questions or plans, never silently decide | Deterministic fallback only |
| Migration workflow | Manage target state, waves, owners, approvals, exceptions, and verification | Local deterministic reference workflow implemented |
| Reporting and API | Expose coverage, evidence maturity, exposure, work, and audit history | Planned |
| Execution adapter | Apply narrowly approved changes with rollback and evidence | Out of scope until governance gates pass |

# 4. Trust zones




The current implementation occupies local equivalents of Zones 1-3 and has no production authority.

# 5. Principal data flows

## 5.1 Observation to inventory

1. Collector reads an authorised, bounded source.
2. Intake validates source envelope and payload schema.
3. Evidence identity and canonical digest are calculated.
4. Replay is classified as new, duplicate, or conflict.
5. Source-specific evidence is normalised without erasing provenance.
6. Inventory record and assessment are created.
7. Observation, outcome, inventory, and assessment commit atomically.

### 5.2 Observation to alert

1. Canonical certificate event is validated.
2. DNS identities are normalised.
3. Versioned policy evaluates the observation.
4. Deterministic severity, score, and alert identifier are created.
5. Advisory explanation is attached without decision authority.
6. Alert is validated and delivered to a sink.

### 5.3 Inventory to migration work (target)

1. Evidence is reconciled with authoritative asset and ownership sources.
2. Dependency paths and cryptographic functions are resolved.
3. Policy proposes priority and identifies missing evidence.
4. Owner selects a supported target state and verification plan.
5. Approvers accept the work, risk, exception, or deferral.
6. Change occurs in an external controlled delivery process.
7. Pre/post tests and new discovery evidence verify the outcome.
8. Coverage, residual risk, and regression status are recalculated.

## 6. Data domains

The target model has separately governed domains:

- **Observation:** what a source reported or a parser derived.
- **Evidence:** provenance, maturity, verification, integrity, scope, and time.
- **Inventory:** a cryptographic use linked to an asset or service.
- **Dependency:** a directional relationship that can block or constrain change.
- **Policy finding:** deterministic evaluation against a versioned policy.
- **Recommendation:** advisory output that requires review.
- **Migration item:** governed work with target, owner, dates, dependencies, and verification criteria.
- **Decision:** approval, rejection, exception, or risk acceptance by an actor.
- **Verification:** evidence that an approved result was achieved.

These domains must not be collapsed into a single mutable status record.

# 7. Deployment view

## Current reference deployment

- Local Python CLI processes.
- Local JSON fixtures and generated certificates.
- SQLite evidence state.
- JSONL/file output.
- Loopback-only TLS connection.
- No credentials, tenant boundary, public API, or production service.

## Target production deployment

- Collectors isolated by source and network scope.
- Authenticated ingestion endpoint with tenant binding and quotas.
- Sandboxed parsers for hostile artifacts.
- Durable event stream and immutable evidence/object storage.
- Transactional inventory and workflow stores.
- Separate policy, advisory, and approval services.
- Identity-aware API and user interface.
- Tamper-evident audit export and monitored SIEM integration.
- Optional execution adapters isolated by environment and privilege.

No cloud or platform choice is fixed by this HLD; selection requires scale, residency, integration, cost, and operating-model requirements.

# 8. Availability and recovery concept

Production readiness requires:

- per-source checkpoints and replay-safe ingestion;
- retry with bounded backoff and dead-letter handling;
- transactional outbox for downstream delivery;
- backup and point-in-time recovery for policy, inventory, workflow, and audit;
- evidence-object integrity checks;
- documented recovery time and recovery point objectives;
- degraded operation that stops execution before it loses auditability.

The local SQLite store demonstrates semantics, not these service levels.

# 9. Security architecture

- Source authentication and tenant binding occur before high-assurance evidence

labels are accepted.

- Parser processes receive no cloud or cryptographic-administration credentials.
- Evidence is append-oriented; corrections are new linked assertions.
- Policy changes use review, versioning, effective dates, and rollback.
- AI outputs are untrusted content with structured validation and citations.
- Human approval is identity-bound and separated from request creation.
- Execution credentials are narrow, expiring, environment-scoped, and absent

from analysis services.

- Sensitive infrastructure metadata is encrypted and field-authorised.

## 10. Key architecture decisions

1. Contract-first deterministic monitoring slice - ADR-001.
2. Replay-safe evidence intake before live collection - ADR-002.
3. Inventory and PQC assessment remain separate artifacts - ADR-003.
4. Real controlled discovery without misrepresenting lab data - ADR-004.
5. Deterministic migration governance without execution authority - ADR-005.
6. Safe reconciliation, bundle, pilot, tenancy, and integration foundations - ADR-006.

## 11. Architecture limitations

The current architecture does not prove complete discovery, chain validation, runtime KEM negotiation, application compatibility, supplier readiness, scale, tenant isolation, service availability, or production security. Those are explicit future validation gates.

## 12. Version 0.9 reference-release components

The `roadmap` component adds four bounded layers: evidence reconciliation and offline exchange; pilot-authorisation checks; tenant-scoped workflow/audit/ outbox persistence; and proposal-only integration records. Trust remains asymmetric: an offline signer can protect integrity without proving observation truth, and an approved proposal conveys no execution authority.

# Low-Level Design

## 1. Scope

This LLD documents the implemented version 0.9.0 reference slice and marks future interfaces as proposed. It should be updated whenever a schema version, security invariant, persistence rule, or public command changes.

## 2. Package structure

| Module | Implemented responsibility |
|---|---|
| `models.py` | Strict domain parsing for certificate events and monitoring policy; evidence-claim enforcement |
| `schema_validation.py` | Cached JSON Schema Draft 2020-12 validation with format checking |
| `validation.py` | Exact-key, string, UUID, timestamp, and list guards |
| `normalization.py` | DNS/IDNA normalisation and policy matching |
| `pipeline.py` | CERT-001 enrichment, deterministic alert decision, severity, score, and explanation |
| `ct_collector.py` | Bounded decoded-fixture pages, provenance, replay, checkpoints, inventory creation |
| `state.py` | SQLite schema and atomic outcome/inventory persistence |
| `inventory.py` | Inventory projection and deterministic PQC exposure assessment |
| `discovery_lab.py` | Synthetic X.509 generation, PEM/DER parsing, and loopback TLS observation |
| `migration.py` | Catalogue validation, dependency ordering, migration planning, decisions, exceptions, verification, and separate metrics |
| `sinks.py` | JSONL development alert sink |
| `*_cli.py` | File-oriented command entry points |

## 3. Contract catalogue

| Contract | Version | Producer/consumer |
|---|---|---|
| Canonical certificate event | 1.3 | Collectors and parsers / pipeline and inventory |
| CT observation page | 1.1 | Fixture source / CT collector |
| Monitoring policy | 1.1 | Policy owner / monitoring pipeline |
| Detection alert | 1.2 | Pipeline / JSONL or future SIEM adapter |
| Cryptographic inventory record | 1.1 | Inventory service / state and future graph |
| PQC assessment | 1.1 | Inventory service / future workflow and reporting |

Contracts reject unexpected fields. A producer must make an explicit version upgrade; the implementation does not silently reinterpret older events.

# 4. Certificate event processing

## 4.1 Validation sequence

```
JSON payload
  -> full JSON Schema validation
  -> exact domain keys and enum checks
  -> timestamp and certificate validity ordering
  -> algorithm family and parameter checks
  -> context validation
  -> provenance and evidence-level policy
  -> optional transport validation
  -> immutable CertificateEvent
```

Material constraints include:

- SHA-256 fingerprints and payload digests are 64 hexadecimal characters.
- Timestamps must include timezone information.
- `not_after` must be later than `not_before`.
- X.509 subject may be empty; SAN may carry identity under RFC 5280 semantics.
- KEM families are not valid certificate-signature families.
- Transport key establishment may be `null`; no value is inferred from a

certificate or cipher-suite label.

## 4.2 Evidence-claim policy

| Collector | Allowed evidence level |
|---|---|
| `fixture-generator` | `simulated` |
| `ct-decoded-fixture` | `simulated` |
| `x509-file-parser` | `generated`, `sanitised-capture` |
| `tls-loopback-lab` | `generated` |

`authenticated-production` and `independently-verified` exist in portable schemas but are rejected at runtime until a trusted collector registry exists. An unknown collector cannot self-assert either level.

# 5. Monitoring rule design

## CERT-001: unknown DNS namespace

Inputs:

- certificate subject and SAN DNS names;
- environment;
- monitoring policy exact names and allowed suffixes.

Processing:

1. Lowercase and remove a final DNS dot.
2. Convert Unicode names using IDNA.
3. Permit a wildcard only as the complete leftmost label.
4. Compare suffixes only at DNS label boundaries.
5. Emit no alert when every name is covered.
6. Otherwise assign `high` in production and `medium` elsewhere.
7. Calculate score as a deterministic base plus a bounded increment for additional unmatched names.
1. Generate a UUIDv5 alert identifier from event, rule, policy ID, and policy version.

The result identifies a policy mismatch, not malicious issuance. The SOC playbook requires evidence and ownership verification.

# 6. CT fixture collection

## 6.1 Bounds

The fixture adapter applies file/page and entry-size limits before downstream processing. A future live adapter must separately define authenticated transport, log trust, pagination, proof verification, timeouts, download limits, and backoff.

## 6.2 Replay algorithm

For each source record:

1. Canonicalise raw JSON bytes.
2. Calculate SHA-256.
3. Query `(collector_id, source_record_id)`.
4. If absent, process and store.
5. If the stored digest matches, classify as duplicate and emit no second alert.
1. If it differs, raise a provenance conflict and stop.
2. Advance the page checkpoint only after every page entry has an outcome.

Rejected domain events are preserved as rejected evidence. Malformed pages fail before state mutation.

# 7. Persistence model

The SQLite reference store maintains:

- collector checkpoint by collector ID;
- observation identity and raw/canonical payload;
- payload digest and processing status;
- canonical event or validation error;
- alert, inventory record, and assessment where produced;
- processing timestamp.

Observation outcome, inventory record, and assessment are committed in one transaction. JSONL delivery is not atomic with SQLite; production requires a transactional outbox.

## 8. Inventory projection

For one certificate event, `PqcInventoryService` creates a stable UUIDv5 record for the X.509 public-key use. It preserves:

- observation identity and time;
- asset/service/business context without inventing missing values;
- function, protocol, implementation placeholders, and algorithm;
- certificate identity, validity, usages, and signature algorithm;
- optional transport observation;
- evidence confidence, evidence level, source evidence, and event provenance;
- a `certificate-only` scope and explicit limitations.

The current projection does not merge multiple observations into a golden asset record. That requires correlation policy and conflict lineage.

## 9. PQC assessment algorithm

Inputs are the public-key family, certificate-signature family, inventory context, and evidence. The algorithm is deterministic:

| Condition | Exposure result |
|---|---|
| RSA, elliptic-curve, or finite-field DH is observed | `quantum-vulnerable` |
| Any relevant family is unknown/other and no known vulnerable family exists | `unknown` |
| Hybrid is observed without deployment proof | `hybrid-unverified` |
| Only recognised PQC families are observed | `candidate-quantum-resistant` |

Known vulnerability dominates unknown evidence. Recognised PQC names do not prove chain, protocol, implementation, peer, or fallback readiness. Missing business context is returned and blocks final priority.

## 10. X.509 discovery laboratory

### 10.1 Synthetic factory

The factory can generate:

- RSA 2048, 3072, and 4096-bit certificates;
- ECDSA P-256 and P-384 certificates;
- valid, expired, and not-yet-valid periods;
- CA and end-entity profiles;
- SAN and no-SAN cases.

Keys are synthetic test material. They must never be imported into a production store.

### 10.2 File parser

The parser:

1. Rejects files larger than one megabyte before parsing.
2. Accepts PEM or DER.
3. Uses PyCA `cryptography` to derive certificate properties.
4. Maps public-key and signature OIDs into the canonical algorithm model.
5. Extracts key usage, EKU, SAN, validity, issuer, subject, serial, and SHA-256 fingerprint.
1. Does not place an absolute local file path into source evidence.
2. Validates the resulting canonical event before returning it.

### 10.3 Loopback TLS probe

The probe:

- accepts an IP literal, not a host name;
- requires the IP to be loopback;
- limits timeout to ten seconds;
- performs a local TLS client handshake;
- records TLS version and cipher suite;
- explicitly records that the peer was not identity-verified;
- leaves negotiated key establishment `null` because the selected Python API does not expose sufficient direct evidence.

## 11. Command interfaces

| Command | Required inputs | Output |
|---|---|---|
| `crypto-control-plane` | event, policy, output | Zero or one JSONL alert |
| `crypto-control-plane-ct-fixture` | fixture directory, policy, state, output | Collection counts, SQLite evidence, alerts |
| `crypto-control-plane-inventory` | event, output | Inventory and assessment JSON bundle |
| `crypto-control-plane-x509-lab` | certificate, evidence level, output | Parsed event, inventory, assessment bundle |
| `crypto-control-plane-migration-plan` | enterprise catalogue, output | Catalogue, work items, and separate coverage/progress report |

Commands return non-zero on invalid input or an unsupported evidence claim. They are local development interfaces, not stable production APIs.

## 12. Implemented Milestone 5 data types

Milestone 5 introduces separately versioned contracts for the following domains.

### Enterprise catalogue record

- `asset_id`, `service_id`, `owner_id`, `environment`
- business criticality, data classification, protection lifetime
- catalogue identity/version, scope, and effective time
- supplier/product/version
- dependency references, cryptographic uses, evidence status, and target state

### Migration work item

- stable work ID and linked inventory/dependency IDs
- current and target cryptographic state by function/protocol
- wave, owner, requester, approver, readiness status, and object version
- blockers, dependencies, verification criteria, and evidence references

### Decision record

- decision ID, type, actor identity, role, time
- object and version decided upon
- rationale, evidence references, conditions, and expiry

### Exception record

- exception ID and exact work-item version
- risk owner, approver, rationale, approval and expiry times
- compensating controls and review cadence

### Verification record

- approved target and change reference
- pre-change and post-change observations
- interoperability and negative-test results
- residual findings and regression checks
- verifier identity and independence level

## 13. Migration state machine

```
planner -> blocked OR approval-required
approval-required -> approved OR rejected OR blocked (time-bounded deferral)
blocked -> excepted (separate active exception record)
approved -> external implementation evidence -> verification-required
verification-required -> verified OR regressed OR verification-required
```

Implemented transitions are policy-checked and reject stale object versions. Production identity binding and an append-only durable audit service remain future controls.

## 14. Error-handling rules

- Contract errors: reject without partial downstream output.
- Replay conflicts: fail closed and preserve prior accepted evidence.
- Individual domain failures in a valid fixture page: store rejected outcome.
- Page envelope failure: no page state changes.
- Sink failure: report failure; production design must retry from an outbox.
- Unsupported evidence level: reject rather than downgrade silently.
- Missing business context: preserve exposure but withhold final priority.

## 15. Observability requirements for production

Metrics should include intake rate, schema failures, replay duplicates, conflicts, checkpoint lag, parser latency, sink backlog, assessment counts, unknown-field rates, evidence age, coverage gaps, workflow ageing, expired exceptions, and verification regressions. Logs must use correlation IDs and avoid leaking full certificate identities or infrastructure metadata by default.

## 16. Implemented Milestones 6 to 9 reference types

- `EvidenceReconciler` groups inventory observations by asset, function, and protocol; retains all source IDs; deterministically identifies the latest record; and emits explicit conflict lineage.
- `OfflineEvidenceBundle` sanitises selected identity fields, hashes canonical artifacts, signs the manifest with Ed25519, and verifies names, sizes, hashes, and signature. Evidence maturity is never derived from signer trust.
- `PilotGate` validates a time-bounded read-only scope, independent approval, source-authentication consistency, and denominator-based coverage.
- `ControlPlaneStore` demonstrates tenant-qualified keys, expected-version updates, separation of duties, audit, and outbox writes in one SQLite transaction.
- `IntegrationProposalService` produces approval-required records with no execution authority and rejects cryptographic or deployment mutation verbs.
- `roadmap_cli` builds the complete local reference evidence set and writes `production_gates_passed: false`.

# Requirements Traceability Matrix

## 1. Purpose

This matrix links requirements to design, implementation, and executable verification. A row marked planned is a controlled gap, not evidence of completion.

## 2. Implemented and partial requirements

| Requirement | Status | Design/code evidence | Test evidence |
|---|---|---|---|
| FR-001 Versioned validated observations | Implemented | `schemas/`, `SchemaRegistry`, `CertificateEvent` | `test_published_examples_conform_to_full_schemas`, `test_legacy_certificate_contract_requires_explicit_upgrade` |
| FR-002 Preserve provenance | Implemented for fixtures/lab | `CertificateEvent.provenance`, `CertificateTransparencyCollector` | `test_collects_pages_preserves_evidence_and_alerts` |
| FR-003 Reject changed-content replay | Implemented | `CollectorState.check_duplicate` | `test_replay_is_idempotent_and_emits_no_second_alert`, `test_source_identity_reuse_with_changed_content_fails_closed` |
| FR-004 Derive X.509 properties | Implemented in lab | `X509CertificateDiscovery` | `test_golden_generated_corpus`, `test_rsa_and_ecdsa_parameters_are_derived_from_certificate` |
| FR-005 Separate evidence domains | Partial | Optional event `transport`, inventory `transport_observation` | `test_loopback_probe_records_tls_without_inventing_key_establishment` |
| FR-006 Carry coverage limits | Implemented for certificate inventory | `PqcInventoryService._record` | `test_classical_certificate_is_quantum_vulnerable_and_high_priority` |
| FR-010 Stable linked inventory | Implemented | UUIDv5 record, observation ID | `test_assessment_is_deterministic`, CT collector tests |
| FR-011 Separate function and algorithm | Implemented for certificate uses | `cryptographic_use.function` and `.algorithm` | Inventory schema and inventory tests |
| FR-012 Identify classical exposure | Implemented | `QUANTUM_VULNERABLE`, assessment rules | `test_classical_certificate_is_quantum_vulnerable_and_high_priority` |
| FR-013 PQC/hybrid requires verification | Implemented | Assessment findings PQC-005/PQC-006 | `test_pqc_algorithms_are_candidates_not_declared_safe`, `test_hybrid_label_requires_protocol_verification` |
| FR-014 Preserve unknown | Implemented | Assessment algorithm | `test_unknown_algorithm_never_becomes_quantum_safe` |
| FR-015 Block unsupported priority | Implemented | Missing-context and priority logic | `test_missing_business_context_blocks_prioritisation` |
| FR-016 Enterprise correlation | Implemented for local evidence and fictional catalogue | `EvidenceReconciler`, catalogue dependency validation | Reconciliation, conflict, unknown-dependency, and cycle tests |
| FR-020 Function-specific target state | Implemented for reference policy | Target compatibility in `MigrationPlanner` | `test_functionally_invalid_supported_target_is_blocked` |
| FR-021 Migration waves | Implemented for simulated catalogue | Deterministic priority and wave assignment | `test_fictional_catalogue_produces_dependency_aware_work` |
| FR-022 Governed work items | Partial local implementation | Versioned work-item contract and `MigrationWorkflow` | Planning, stale-decision, and verification tests |
| FR-023 Exceptions | Implemented locally | Separate expiry/risk-owner/compensating-control contract | Active and expired exception tests |
| FR-024 Change approval/verification | Implemented locally without execution | Decision and verification contracts; separation checks | Self-approval, stale decision, and complete verification tests |
| FR-025 Regression detection | Implemented for local evidence | Failed verification produces `regressed`; reconciliation preserves contradictory post-change states | Verification and reconciliation tests |
| FR-026 Coverage reporting | Implemented for explicit catalogue scope | Separate coverage/progress report | `test_coverage_and_migration_progress_are_separate` |
| FR-030 Deterministic detection | Implemented | `CertificateMonitoringPipeline` | Pipeline test suite |

| Requirement | Status | Design/code evidence | Test evidence |
|---|---|---|---|
| FR-031 Evidence-linked explanation | Partial | Deterministic explanation and alert evidence | `test_unknown_domain_produces_high_severity_alert` |
| FR-032 AI cannot change decisions | Implemented structurally | No model call; explainer receives decided result | Determinism and alert schema tests |
| NFR-001 Fail closed | Implemented for local interfaces | Exact-key/domain/schema validation | Invalid fingerprint, timestamp, field, wildcard, file-size, and malformed-certificate tests |
| NFR-002 Auditability | Partial | Provenance, policy version, deterministic IDs, SQLite state | CT collection and deterministic-ID tests |
| NFR-006 Portable contracts | Implemented | JSON Schema Draft 2020-12 | Full-schema tests |
| NFR-007 Reproducible assessment | Implemented | Deterministic rules | `test_assessment_is_deterministic`, `test_alert_identifier_is_deterministic` |
| FR-033 Controlled delivery | Partial local reference | Transactional SQLite audit and outbox; proposal-only adapters | Control-plane transaction and proposal tests |
| NFR-005 Isolation | Partial local reference | Tenant-qualified storage, role and separation checks | Tenant, role, self-approval, and stale-version tests |
| NFR-008 Testability | Implemented for current scope | `tests/` | 75 automated tests |

## 3. Planned requirements and required proof

| Requirement | Status | Required implementation evidence | Required acceptance evidence |
|---|---|---|---|
| FR-007 Multi-source collectors | Planned | Authenticated source adapters and conformance kit | Authorised integration tests; reconciliation and coverage report |
| NFR-003 Privacy | Planned beyond local use | Data classification, encryption, redaction, retention | Privacy review and access-control tests |
| NFR-004 Availability | Planned | Service SLOs, queueing, recovery design | Load, failover, restore, and game-day evidence |
| NFR-009 Scalability | Planned | Distributed deployment and partition model | Representative performance and capacity results |
| NFR-010 Accessibility | Planned | User research and accessible UI | Accessibility audit and task-completion study |

## 4. Security invariant traceability

| Invariant | Implementation | Evidence |
|---|---|---|
| Invalid input produces no alert | Validation precedes pipeline decision | Pipeline negative tests |
| Explanation cannot create/change alert | Decision calculated before explainer call | Pipeline structure and deterministic tests |
| DNS suffixes match label boundaries | `matches_policy` | Boundary and IDNA tests |
| Source identity cannot change bytes silently | Digest conflict check | CT provenance-conflict test |
| Unknown is never quantum-safe | Assessment decision order | Unknown and mixed-evidence tests |
| Observation/inventory/assessment commit together | SQLite transaction | CT persistence tests |
| Generated/fixture evidence cannot claim production | Evidence allowlist and reserved levels | Evidence escalation tests |
| TLS lab cannot reach external targets | Loopback IP validation | `test_loopback_probe_refuses_external_target` |
| Cipher/certificate does not invent KEM | Nullable transport field | Loopback TLS test |
| Dependency graph fails closed | Topological validation and identity checks | Cycle, unknown dependency, and conflicting ownership tests |
| Requester cannot self-approve | Version-bound workflow guard | Self-approval and stale-decision tests |
| Exceptions expire | Exception time and state validation | Active and expired exception tests |
| Verification requires independent pre/post evidence | Verification target, actor, criteria, and evidence gates | Complete/incomplete verification tests |
| Coverage is not completion | Separate report objects and percentages | Separate-metrics test |
| Conflicts are not silently resolved | Reconciliation lineage and conflict state | Roadmap reconciliation tests |
| Signing does not upgrade evidence | Separate evidence claim and signer trust | Bundle trust-boundary tests |
| Bundle tampering fails closed | Manifest names, sizes, SHA-256, and Ed25519 | Bundle tamper test |
| Pilot scope is authorised and current | Read-only scope, expiry, independent approver | Pilot negative tests |
| Tenant data is isolated | Tenant-qualified storage and lookup | Tenant snapshot test |
| Workflow updates are versioned | Expected-version transition | Stale workflow test |
| Integrations cannot execute changes | Proposal schema and blocked mutation verbs | Integration proposal test |

## 5. Traceability maintenance rule

A change is incomplete if it adds or materially changes a mandatory requirement without updating:

1. its identifier and acceptance criterion;
2. the HLD/LLD or an ADR;
3. its contract and compatibility statement;
4. automated or manual verification evidence;
5. the known limitation or risk entry where applicable.

# Verification and Validation Plan

## 1. Objective

Verification asks whether the system was built according to its requirements and design. Validation asks whether it is suitable for the intended user and operational problem. The current repository provides substantial verification for a local reference slice and limited validation of the broader product need.

## 2. Test levels

| Level | Purpose | Current evidence |
|---|---|---|
| Contract | Reject malformed or incompatible data | Full JSON Schema and domain tests |
| Unit | Verify normalisation, decisions, mappings, and failure rules | Python `unittest` suite |
| Component | Exercise collector, state, inventory, parser, and CLI boundaries | CT, inventory, discovery tests |
| Integration | Exercise multi-module local paths | Fixture-to-state/alert/inventory tests |
| Cryptographic artifact | Parse real generated PEM/DER certificates | Golden X.509 corpus |
| Protocol | Observe a real bounded TLS handshake | Loopback TLS test |
| Security negative | Demonstrate refusal and fail-closed properties | Oversize, malformed, replay conflict, evidence escalation, external target tests |
| System/operational | Validate deployed service behavior | Not yet available |
| User acceptance | Validate analyst and migration-owner workflows | Not yet available |
| Independent assurance | External review or penetration test | Not yet available |

## 3. Current automated acceptance suite

The baseline has 75 tests covering:

- schema and format conformance;
- DNS policy normalisation and boundary behavior;
- deterministic alerts and assessments;
- invalid-input rejection;
- CT pagination, checkpoint restart, replay, provenance conflict, and size bounds;
- inventory classification, unknown handling, hybrid/PQC verification status,

missing business context, and function correctness;

- real generated RSA/ECDSA X.509 parsing;
- validity, CA, SAN/no-SAN, PEM/DER, malformed, and oversized cases;
- evidence-maturity enforcement;
- loopback TLS observation and non-loopback refusal;
- CLI output bundles.
- fictional catalogue, ownership, dependency, target, priority, wave, decision,

exception, verification, and separate-metric behavior.

Supported verification environments are Python 3.11-3.13 in project metadata and CI intent. The latest local evidence records complete passing runs on Python 3.12 and 3.13. A clean run should be attached to every release candidate.

## 4. Reproduction procedure

```
python3 -m venv .venv
.venv/bin/python -m pip install -e .
.venv/bin/python -m unittest discover -s tests -v
git diff --check
```

Record:

- commit SHA and branch;
- operating system and Python version;
- dependency resolution or lock evidence;
- start/end time;
- passed, failed, errored, and skipped counts;
- complete failure output;
- reviewer identity where evidence is used for a gate.

## 5. Golden-scenario validation

`examples/lab/scenarios.json` is the controlled expected-result manifest. Changes require review because a changed expectation can hide a regression. Golden scenarios must include at least:

- RSA 2048 valid end entity;
- ECDSA P-256 valid end entity;
- RSA 3072 expired certificate;
- ECDSA P-384 not-yet-valid certificate;
- RSA 4096 CA without SAN.

Future migration scenarios should add services, owners, data lifetimes, dependencies, target states, blockers, exceptions, and expected wave outcomes.

## 6. Production-readiness validation gates

### Gate A: authenticated pilot intake

- Written authorisation and target scope.
- Collector identity bound to tenant and source.
- TLS and credential handling reviewed.
- Bounded collection, timeout, retry, and stop controls tested.
- Raw evidence protected and retention approved.
- Coverage denominator supplied by an authoritative source.

### Gate B: inventory quality

- Sample records independently compared with source systems.
- False merge and missed merge rates measured.
- Unknown and stale evidence rates reported.
- Asset, owner, and dependency conflicts remain visible.
- No unsupported completeness claim.

### Gate C: migration workflow

- Role and separation-of-duties matrix approved.
- State transitions and stale-write prevention tested.
- Exception expiry and escalation demonstrated.
- Before/after verification and regression paths exercised.
- Audit export reviewed by a non-developer stakeholder.

### Gate D: production service

- Threat model and data-protection assessment updated.
- Dependency and container scanning completed.
- Penetration test findings resolved or accepted.
- Backup restore, failover, and incident runbooks exercised.
- Monitoring, on-call ownership, SLOs, and support model approved.

### Gate E: optional execution

- Business need proves that integrated execution is preferable to an external

change system.

- Narrow adapter design and environment boundaries approved.
- Dual control, just-in-time access, dry run, rollback, and kill switch tested.
- Analysis and AI services have no access to execution credentials.
- Independent security approval obtained.

## 7. Test data strategy

Use data in increasing order of assurance:

1. Hand-authored simulated fixtures for rule edge cases.
2. Generated cryptographic artifacts for parser and protocol behavior.
3. Explicitly sanitised captures for realistic variation.
4. Authenticated production observations in an authorised pilot.
5. Independently verified samples for assurance measurement.

Never relabel a lower level. Sanitisation must preserve the property under test, remove secrets and unauthorised identifiers, document transformations, and retain source approval outside the repository.

## 8. Milestone 5 verification result

1. Fictional enterprise catalogue and dependency graph contracts validate.
2. Migration work, decision, exception, and verification contracts validate.
3. Deterministic wave assignment handles required context and unknowns.
4. Cycles, conflicting ownership, unsupported targets, stale decisions, expired exceptions, and missing verification fail safely.
1. Coverage and migration completion are reported separately.
2. Existing tests remain passing and the new requirements have tests.
3. ADR, threat model, PRD, LLD, and traceability matrix are updated.

## 9. Milestones 6 to 9 reference verification result

1. Reconciliation is deterministic, retains lineage, and reports conflicts.
2. Sanitisation preserves algorithm properties and does not upgrade evidence.
3. Offline bundle round-trip verification passes and artifact tampering fails.
4. Trusted and self-signed signer states remain separate from evidence level.
5. Pilot scope, expiry, self-authorisation, and out-of-scope records fail safely.
6. Tenant-scoped workflow writes audit and outbox records transactionally.
7. Role, separation-of-duties, and expected-version controls fail closed.
8. Proposal-only integrations reject mutation operations.
9. The complete suite contains 75 tests; production gates remain explicitly false because this is simulated local evidence.

# Security, Risk and Assurance Plan

## 1. Assurance position

The current implementation demonstrates controlled engineering properties. It does not demonstrate production security, full discovery coverage, compliance, or successful migration of a real organisation.

Permitted current claims:

- versioned evidence and assessment contracts exist;
- deterministic reference rules are executable and tested;
- generated X.509 artifacts are parsed using a real cryptographic library;
- replay, conflict, evidence escalation, malformed input, and external target

refusal have automated tests;

- current limitations are documented.

Prohibited current claims:

- the product has discovered an enterprise estate;
- a customer or organisation is quantum-safe;
- the product validates complete TLS or PKI security;
- the product is certified, independently audited, or production hardened;
- its AI autonomously manages cryptography;
- its current evidence is authenticated production evidence.

## 2. Assets requiring protection

- Source observations and their provenance.
- Cryptographic inventory and dependency intelligence.
- Asset identities, owners, classifications, and data lifetimes.
- Policies, assessments, migration targets, and exception records.
- Approval identities and audit history.
- Future collector and execution credentials.
- Availability and integrity of migration-status reporting.

## 3. Primary threat scenarios

| Scenario | Consequence | Current or required control |
|---|---|---|
| Forged or mislabelled evidence | Incorrect exposure or false completion | Strict contracts now; authenticated collector registry required |
| Missing source represented as complete coverage | Hidden vulnerable systems | Explicit coverage limits; authoritative denominator required |
| Correlation merges unrelated assets | Wrong owner or unsafe migration | Source lineage, conflict representation, review thresholds |
| Algorithm label represented as protocol proof | False PQC assurance | Function-specific evidence and verification-required states |
| Policy tampering | Suppressed work or manipulated priority | Version, approval, signature, effective date, rollback |
| AI prompt or evidence injection | Misleading advice or unsafe action | Treat output as untrusted; deterministic decisions; no credentials |
| Privilege convergence | One compromise can decide and execute | Separate collector, analysis, approval, and execution identities |
| Exception never expires | Permanent unmanaged exposure | Mandatory expiry, owner, approver, escalation, reporting |
| Sensitive inventory disclosure | High-value infrastructure intelligence leaks | Encryption, tenant isolation, field authorisation, redaction, audit |
| Dependency/supplier change | Verified item silently regresses | Continuous reconciliation and regression state |
| Parser or fixture resource abuse | Denial of service | Bounds now; sandbox, quota, and isolation required |
| Audit deletion or rewrite | Inability to prove decisions | Append-oriented records and tamper-evident external export |

The detailed repository threat model remains the authoritative technical threat inventory for implemented milestones.

## 4. Control objectives

### Evidence trust

- Authenticate collector workload and source account.
- Bind evidence to tenant, collector, scope, time, and software version.
- Use digest/signature semantics accurately; a hash is not authentication.
- Prevent an unknown collector from self-asserting a high assurance level.
- Retain raw evidence or a defensible reference and transformation history.

### Decision integrity

- Version policy and record the exact version used.
- Make priority and transition gates deterministic.
- Require reasoned, named approval for exceptions and material changes.
- Prevent requesters from approving their own high-risk work.
- Reject decisions against stale object versions.

### AI governance

- Limit AI to evidence correlation, summarisation, drafting, and options.
- Require structured outputs with source references and confidence.
- Evaluate hallucination, omission, injection, and inconsistent recommendation.
- Never give a model direct CA, HSM, KMS, trust-store, or deployment authority.
- Log accepted, rejected, and edited recommendations for evaluation.

### Execution safety

- Prefer external change systems until integrated execution is justified.
- If added, isolate each adapter and environment.
- Use narrow, short-lived credentials and explicit approved change payloads.
- Provide dry-run, idempotency, rollback, rate limits, and a kill switch.
- Require independent post-change evidence before marking verified.

## 5. Risk register

| ID | Risk | Likelihood | Impact | Current disposition |
|---|---|---|---|---|
| R-001 | Lab success mistaken for production readiness | Medium | High | Mitigated by evidence levels and claim boundaries |
| R-002 | Certificate discovery misses non-certificate cryptography | High | High | Accepted for current scope; multi-source roadmap required |
| R-003 | Missing ownership blocks prioritisation | High | High | Explicitly blocks final priority; catalogue planned |
| R-004 | Dependency graph is incomplete or stale | High | High | Planned reconciliation and confidence model |
| R-005 | PQC support differs across protocol peers/products | High | High | Verification required; no algorithm-only completion |
| R-006 | SQLite/JSONL used beyond reference scale | Medium | High | Architecture limitation; replace before production |
| R-007 | Sensitive discovery data is exposed | Medium | High | Local-only now; privacy controls required before pilot |
| R-008 | AI scope expands into ungoverned automation | Medium | Critical | Explicit authority boundary and future approval gate |
| R-009 | Standards or supplier roadmaps change | High | Medium | Versioned policy, target state, and periodic review |
| R-010 | Documentation drifts from code | Medium | Medium | Traceability and release evidence review |

Risk owners, target dates, treatment cost, and residual ratings must be assigned before an authorised pilot.

## 6. Assurance artefacts by gate

| Gate | Required evidence |
|---|---|
| Reference release | Source, schemas, tests, ADRs, threat model, traceability, reproducible run |
| Pilot approval | Scope/authorisation, data assessment, architecture review, operating model, rollback/stop procedure |
| Production approval | Secure build evidence, access model, pen test, recovery test, monitoring, incident plan, residual-risk acceptance |
| Migration completion | Approved target, change reference, interoperability tests, post-change discovery, independent verification where required |

## 7. Compliance mapping approach

Do not claim compliance from this repository alone. When a target organisation and jurisdiction are known:

1. identify applicable obligations and internal control framework;
2. map control objectives to system capabilities and operating procedures;
3. record inherited, shared, and customer responsibilities;
4. attach objective evidence and test frequency;
5. identify gaps and approved remediation;
6. obtain qualified legal, compliance, and audit review where required.

## 8. Review cadence

- Every schema or security-boundary change: update LLD, traceability, and threat

model in the same change.

- Every release: reproduce tests and archive the evidence manifest.
- Quarterly during development: review risk register and standards assumptions.
- Before each pilot or new source: perform source-specific threat modelling.
- Before execution capability: conduct an independent architecture and security review.

# Delivery Roadmap and Governance

## 1. Completed foundation

| Milestone | Outcome | Evidence |
|---|---|---|
| 1 | Contract-first certificate monitoring vertical slice | Commit `bfdc4ff`, ADR-001, pipeline tests |
| 2 | Replay-safe bounded intake and durable evidence | Commit `20e5705`, ADR-002, CT tests |
| 3 | Cryptographic inventory and deterministic PQC exposure | Commit `887fdb3`, ADR-003, inventory tests |
| 4 | Controlled real X.509 and loopback TLS discovery lab | Commit `be71182`, ADR-004, discovery tests |
| 5 | Deterministic migration planning and governance using a fictional enterprise | ADR-005, migration schemas, planner/workflow tests |
| 6 | Conflict-preserving reconciliation and integrity-protected offline evidence | ADR-006, roadmap module, bundle tests |
| 7 | Read-only pilot authorisation and measurement reference gate | ADR-006, pilot schema and negative tests |
| 8 | Tenant-scoped workflow, audit, and outbox reference foundation | ADR-006, SQLite transaction and isolation tests |
| 9 | Proposal-only integration boundary | ADR-006, proposal schema and mutation rejection tests |

These commits are local project evidence unless and until deliberately pushed and reviewed on GitHub.

## 2. Reference implementation status and external gates

### Milestone 6 - Evidence reconciliation and offline bundle: locally implemented

- multi-observation correlation with conflict lineage;
- CBOM-style export and dependency graph views;
- sanitisation tool and customer-controlled offline evidence bundle;
- signed manifest, import validation, and data-quality report;
- report generation for technical and executive audiences.

Local exit met with simulated/generated data. External trust establishment and customer acceptance remain open.

### Milestone 7 - Authorised read-only pilot: reference gate implemented

- select one bounded source and obtain written authorisation;
- implement collector authentication and scope policy;
- add authoritative asset denominator and ownership enrichment;
- operate read-only, measure quality, and report unknowns;
- perform privacy and security review.

Real exit open: written authority, bounded source, authenticated collection, privacy/security review, and measured real-world error rates are absent.

### Milestone 8 - Production control-plane foundation: local reference implemented

- tenant and identity architecture;
- durable stream, evidence storage, transactional workflow, and outbox;
- UI/API, RBAC, separation of duties, audit export;
- SLOs, recovery, observability, secure delivery pipeline;
- independent security testing.

Real exit open: no approved production identity, service, SLO, recovery test, secure-delivery evidence, or independent security assessment exists.

### Milestone 9 - Optional controlled integrations: proposal boundary implemented

- integrate ticketing, CMDB, PKI, KMS/HSM, and deployment evidence;
- add narrowly scoped action proposals or external change triggers;
- consider isolated execution only after a separate risk decision.

Real exit open: no organisational adapter is connected or authorised. Every reference proposal has no execution authority.

## 3. Governance roles

| Role | Accountable decision |
|---|---|
| Product owner | Product outcomes, scope, and user acceptance |
| Security/cryptography authority | Algorithm policy and target-state approval |
| Enterprise architect | System and dependency architecture |
| Data owner | Collection purpose, classification, retention, and access |
| Service owner | Migration plan, implementation, and operational acceptance |
| Change authority | Production change approval and scheduling |
| Risk owner | Exception and residual-risk acceptance |
| Independent verifier | Evidence sufficiency for high-risk completion |
| Engineering team | Implementation, tests, operations, and defect resolution |

One person may perform multiple roles in a prototype, but the production design must identify incompatible duties and compensating review.

## 4. Decision gates

| Gate | Decision | Minimum evidence |
|---|---|---|
| G0 Concept | Continue problem discovery | Problem statement and stakeholder need |
| G1 Reference architecture | Accept design direction | PRD, HLD, threat model, ADRs |
| G2 Reference implementation | Accept technical feasibility | LLD, source, schemas, tests, evidence guide |
| G3 Pilot | Permit bounded real data | Authorisation, source scope, privacy/security review, stop plan |
| G4 Production | Operate as a service | SLOs, access model, recovery, monitoring, pen test, runbooks |
| G5 Migration execution | Permit change authority | Separation of duties, adapter review, dry run, rollback, independent approval |

Passing one gate does not imply the next.

## 5. Delivery controls

- Use short-lived branches and reviewed pull requests.
- Require tests, schema compatibility review, threat-model delta, and traceability update for material changes.
- Pin external build actions and dependencies where practical.
- Produce a release evidence manifest with commit, environment, tests, schema versions, known issues, and artifact hashes.
- Sign releases and produce an SBOM before pilot distribution.
- Never mix secrets, customer evidence, or synthetic private keys into source control.

## 6. Backlog priority method

Prioritise features using:

1. assurance or safety risk reduced;
2. evidence needed for the next gate;
3. user value and decision enabled;
4. dependency order;
5. reversibility and learning value;
6. engineering and operating cost.

This places evidence reconciliation before a broad UI and authenticated read-only discovery before any execution authority.

## 7. Change control

Material changes require an ADR when they alter:

- trust boundaries or privilege;
- evidence or assurance semantics;
- canonical contract compatibility;
- data retention/classification;
- deterministic decision authority;
- workflow states and approval rules;
- production deployment or external integrations.

## 8. Immediate external-evidence backlog

1. Obtain written authorisation for one bounded read-only source.
2. Approve source authentication, privacy, retention, and stop controls.
3. Establish an external signer trust registry for bundle import.
4. Select and validate production identity and tenant isolation architecture.
5. Demonstrate backup, recovery, observability, SLOs, and secure delivery.
6. Commission independent security testing.
7. Review each integration adapter and its credentials separately.
8. Run the full suite on Python 3.11, 3.12, and 3.13.

# Project Evidence and Demonstration Guide

## 1. Claim being demonstrated

The repository is a working local reference implementation of an evidence-backed cryptographic discovery and PQC assessment foundation. It demonstrates deterministic certificate monitoring, replay-safe evidence intake, certificate-focused inventory and assessment, real generated X.509 parsing, and a bounded loopback TLS observation.

It does not demonstrate production estate coverage or completed PQC migration.

## 2. Evidence baseline

```
git log --oneline --decorate -5
git status --short --branch
git show --stat --oneline be71182
```

Expected milestone history:

```
be71182 Build controlled cryptographic discovery lab
887fdb3 Build PQC inventory and exposure foundation
20e5705 Build trustworthy CT intake milestone
bfdc4ff Build initial certificate monitoring control plane
```

If the current commit differs, record it and review the delta instead of assuming that this guide still describes the code exactly.

## 3. Environment setup

```
python3 --version
python3 -m venv .venv
.venv/bin/python -m pip install -e .
.venv/bin/python -m pip freeze
```

The repository currently pins `cryptography==50.0.2` and `jsonschema[format-nongpl]==4.26.0`.

## 4. Automated proof

```
.venv/bin/python -m unittest discover -s tests -v
git diff --check
```

Expected baseline: 61 passing tests. Do not hide failures or skips. A platform that prevents local socket creation may block the loopback test; run that test in an authorised local or CI environment and record the constraint separately.

## 5. Demonstration A - deterministic certificate alert

```
.venv/bin/python -m crypto_control_plane.cli \
  --event examples/events/unknown-domain-issued.json \
  --policy examples/policy.json \
  --output build/alerts.jsonl
```

Review:

- rule ID `CERT-001`;
- unmatched name evidence;
- policy ID/version;
- deterministic severity, score, and alert ID;
- advisory explanation authority;
- source evidence and provenance.

Then run the known-domain example and confirm it produces no alert.

## 6. Demonstration B - replay-safe evidence intake

```
.venv/bin/python -m crypto_control_plane.ct_cli \
  --fixture-dir examples/ct/pages \
  --policy examples/policy.json \
  --state build/ct-state.sqlite \
  --output build/ct-alerts.jsonl
```

Run it a second time. The same source records should be duplicates and should not produce duplicate alerts. The tests separately demonstrate failure when a source identity is reused with changed content.

## 7. Demonstration C - PQC inventory assessment

```
.venv/bin/python -m crypto_control_plane.inventory_cli \
  --event examples/events/known-domain-issued.json \
  --output build/pqc-inventory-assessment.json
```

Review:

- evidence and observation linkage;
- cryptographic function and algorithm;
- certificate signature as a separate algorithm field;
- `certificate-only` coverage limitation;
- quantum-vulnerable classification for classical public-key evidence;
- missing context and verification requirements.

## 8. Demonstration D - real X.509 parsing

Use the test suite's generated certificate corpus or generate a synthetic certificate through `SyntheticCertificateFactory`. Then run:

```
.venv/bin/python -m crypto_control_plane.discovery_lab_cli \
  --certificate /path/to/generated-certificate.pem \
  --evidence-level generated \
  --output build/x509-discovery-bundle.json
```

Review the derived fingerprint, public-key family/size or curve, signature algorithm/OID/hash, validity, SAN, key usage, EKU, inventory record, and PQC assessment. Confirm that an absolute input path is not leaked into evidence.

## 9. Demonstration E - security boundaries

The most efficient proof is to run the named tests:

```
.venv/bin/python -m unittest \
  tests.test_discovery_lab.DiscoveryLabTests.test_loopback_probe_refuses_external_target \
  tests.test_discovery_lab.DiscoveryLabTests.test_fixture_cannot_be_upgraded_to_production_evidenc
e \
  tests.test_discovery_lab.DiscoveryLabTests.test_unknown_collector_cannot_claim_production_eviden
ce \
  tests.test_ct_collector.CertificateTransparencyCollectorTests.test_source_identity_reuse_with_ch
anged_content_fails_closed
```

These show that the implementation refuses external TLS targets, unsupported evidence escalation, and provenance conflict.

## 10. Reviewer checklist

- [ ] Repository and commit identity recorded.
- [ ] Working tree state recorded.
- [ ] Dependency versions recorded.
- [ ] Complete tests executed and results retained.
- [ ] At least one positive and one no-alert scenario reproduced.
- [ ] Replay/idempotency behavior reproduced.
- [ ] Inventory and assessment output reviewed.
- [ ] Real X.509 parsing demonstrated.
- [ ] Security refusal cases demonstrated.
- [ ] PRD-to-test traceability sampled.
- [ ] Threat model and limitations reviewed.
- [ ] No production, certification, coverage, or migration-completion claim made

without additional evidence.

## 11. Evidence manifest template

```
Review date:
Reviewer:
Repository:
Branch:
Commit SHA:
Operating system:
Python version:
Dependency evidence:
Test result:
Skipped tests and reason:
Demonstrations completed:
Findings:
Known limitations accepted:
Decision: accept / accept with conditions / reject
Decision authority:
```

## 12. Milestone 5 migration-management demonstration

The fictional enterprise scenario now demonstrates:

- discovery records linked to owners and services;
- dependency-aware planning;
- explicit current and target states by cryptographic function;
- deterministic wave and blocker decisions;
- time-bounded exceptions and approval history;
- before/after verification evidence;
- separate coverage and migration metrics;
- safe handling of unknowns, conflicts, cycles, expiry, and regression.

Run it with:

```
.venv/bin/python -m crypto_control_plane.migration_cli \
  --catalog examples/migration/fictional-enterprise.json \
  --output build/migration-plan.json
```

The baseline must remain labelled simulated and must report discovery coverage separately from verified migration progress.

## 13. Milestones 6 to 9 reference-release demonstration

Run:

```
.venv/bin/python -m crypto_control_plane.roadmap_cli \
  --event examples/events/known-domain-issued.json \
  --event examples/events/unknown-domain-issued.json \
  --output-dir build/reference-release
```

Review `summary.json`, `reconciliation.json`, `cbom.json`, `dependency-graph.json`, `offline-bundle.json`, `bundle-verification.json`, `pilot-report.json`, `control-plane-reference.json`, and `integration-proposal.json`.

Acceptance evidence includes verified bundle integrity, untrusted self-signed signer status, simulated pilot status, tenant-scoped audit/outbox counts, no integration execution authority, and `production_gates_passed: false`.

# ADR-001: Contract-first deterministic certificate monitoring slice

- Status: accepted
- Date: 2026-10-11

## Context

The target system may eventually ingest certificate events from ACME, cloud certificate managers, cert-manager, Certificate Transparency, TLS discovery, Vault, HSM-backed services, and AD CS. It may also provide AI-assisted analysis and governed lifecycle workflows. Building all integrations at once would hide trust assumptions and prevent early end-to-end verification.

## Decision

Implement the first slice as a synchronous, dependency-light Python pipeline:

1. Accept a versioned canonical certificate event.
2. Perform strict domain validation and DNS-name normalization.
3. Enrich the event against an explicit organization DNS policy.
4. Evaluate deterministic rule `CERT-001` (unknown DNS namespace).
5. Create a deterministic alert ID and risk score.
6. Add a deterministic, advisory explanation from structured evidence.
7. Append the alert to a local JSONL development sink.
8. Provide a corresponding SOC playbook and automated tests.

The schemas are language-neutral JSON Schema Draft 2020-12 documents. Detection results do not depend on an LLM. No component can issue, renew, revoke, install, or delete certificates or keys.

## Consequences

Positive:

- The first milestone is locally executable and auditable.
- Collector and SIEM adapters can evolve around stable contracts.
- Tests capture security-relevant normalization and policy behavior.
- A future model adapter can be evaluated without changing alert semantics.

Negative:

- The local validator implements the security-critical subset of the published

schemas; independent full JSON Schema conformance tooling is still required.

- JSONL lacks production delivery, integrity, retry, and access controls.
- DNS allowlists do not establish legal or operational domain ownership.
- The synchronous path is not a scale architecture.

## Guardrails for later milestones

- Sign or otherwise authenticate collector-to-control-plane events.
- Define tenancy, replay, idempotency, retention, and redaction policies.
- Treat model output as untrusted advisory content; validate structured output.
- Require deterministic policy and appropriate human approval for high-impact cryptographic operations.
- Isolate any future executor from analysis services and use narrow, expiring, environment-scoped credentials.

# ADR-002: Trustworthy intake contracts and replay-safe fixture collection

- Status: accepted
- Date: 2026-10-11

## Context

Milestone 1 proved the canonical event-to-alert path with hand-authored events. It did not prove full schema conformance, evidence preservation, restart behavior, replay handling, or a collector boundary. Connecting directly to a live CT service at this point would combine unproven parsing, transport, pagination, trust, and operational behavior.

## Decision

Implement a provider-neutral decoded CT fixture source behind a narrow source protocol. Validate every page, event, policy, and alert using JSON Schema Draft 2020-12 plus domain checks. Add a versioned provenance envelope containing the collector identity/version, source-record identity, raw-entry SHA-256, stated verification state, and source cursor.

Persist the following in SQLite:

- collector checkpoints;
- the canonical raw CT entry;
- payload digest;
- canonical event;
- accepted, alerted, or rejected outcome;
- generated alert or validation error; and
- processing time.

Treat a repeated source identity with the same digest as a duplicate. Treat the same source identity with a different digest as a provenance conflict and fail closed. Checkpoint only after a fully processed page. A restart resumes after the last completed page; a partial-page retry is safe because completed entries are idempotent.

## Consequences

Positive:

- Tests exercise pagination, restart, replay, conflict, evidence, and alerting

without external network or credential dependencies.

- Raw evidence and policy versions are linked to detection outputs.
- A future live adapter has a small, testable contract.
- CI validates the supported Python versions with read-only permissions and

immutable action commit pins.

Negative and residual:

- A SHA-256 digest detects changed bytes but does not authenticate the source.
- The decoded fixture contract is not an implementation of CT wire protocols or

cryptographic proof verification.

- SQLite is a single-process reference store, not a distributed event system.
- Appending an alert to JSONL and recording SQLite state are not one atomic

transaction. A production integration needs a transactional outbox and explicit delivery semantics.

- Policy ownership remains illustrative rather than authoritative.

## Rejected alternatives

- **Live CT first:** rejected because transport and parsing failures would mask

contract and replay defects.

- **In-memory deduplication:** rejected because it does not survive restart.
- **Digest described as a signature:** rejected because an unauthenticated hash

provides integrity comparison, not source authenticity.

- **LLM extraction of CT records:** rejected because parsing and detection must

remain deterministic and auditable.

# ADR-003: Evidence-backed cryptographic inventory and PQC exposure

- Status: accepted
- Date: 2026-10-11

## Context

Milestones 1 and 2 proved deterministic certificate alerting and replay-safe evidence intake. The product goal is broader: an evidence-backed cryptographic discovery and PQC migration control plane. Certificate observations are useful discovery evidence, but they do not establish negotiated key establishment, application implementations, business ownership, dependency paths, or complete cryptographic coverage.

## Decision

Introduce two separately versioned and validated artifacts:

1 A cryptographic inventory record that preserves the source observation,

cryptographic function, algorithm details, asset context, evidence confidence, and explicit coverage limitations.

1 A deterministic PQC assessment that classifies known classical public-key

exposure, marks unknown evidence as unknown, treats PQC algorithms as candidates requiring deployment verification, and refuses to finalise migration priority without required business context.

Extend the canonical certificate event to version 1.2 with decoded public-key, certificate-signature, key-usage, extended-key-usage, and optional business context. Persist validated inventory records and assessments atomically with the source processing outcome.

## Safety semantics

- RSA, elliptic-curve, and finite-field DH families are classified as

quantum-vulnerable evidence.

- ML-KEM, ML-DSA, and SLH-DSA evidence is `candidate-quantum-resistant`, not a

declaration of system safety or migration completion.

- A hybrid label is `hybrid-unverified` until the protocol, implementation,

peers, trust chain, and fallback behavior are tested.

- Known vulnerable evidence dominates unknown evidence; an unknown field cannot

hide a known classical exposure.

- Certificate key usage indicates permitted capability, not proof of runtime

protocol negotiation.

- Missing evidence stays explicit and blocks unsupported conclusions.

## Consequences

Positive:

- The same intake foundation now produces auditable discovery and assessment artifacts for the future migration control plane.
- Business context and algorithm exposure are separated, allowing exposure to be identified without inventing migration priority.
- Coverage limitations travel with each record and reduce false confidence.
- Later TLS, code, cloud, KMS, HSM, private-PKI, and vendor collectors can target the same inventory boundary.

Negative and residual:

- Algorithm values are supplied by decoded fixtures; live X.509 parsing and independent verification are not implemented.
- The current inventory covers certificates only and contains no resolved dependency graph.
- CT observations generally lack authoritative owner, service, data-lifetime, implementation, and supplier context.
- SQLite remains a single-process reference store.
- The version 1.2 event contract intentionally requires producers to provide explicit cryptographic evidence or `unknown`; version 1.1 events fail closed.

## Rejected alternatives

- **Treat missing fields as safe:** rejected because absence of evidence is not evidence of quantum resistance.
- **Declare ML-DSA/ML-KEM observations compliant:** rejected because algorithms alone do not establish protocol, chain, implementation, or relying-party readiness.
- **Infer TLS key establishment from the certificate:** rejected because certificate authentication/signatures and negotiated key establishment are distinct evidence domains.
- **Build migration orchestration now:** rejected until discovery coverage, authoritative ownership, policy, approvals, separation of duties, audit, and rollback controls exist.

# ADR-004: Controlled discovery lab without customer data

- Status: accepted
- Date: 2026-10-11

## Context

The project has no authorised customer or production cryptographic dataset. Milestone 3 established inventory and PQC assessment contracts, but decoded fixtures do not validate real certificate parsing or TLS observation. Pausing would delay reusable engineering; treating synthetic evidence as production would create false assurance.

## Decision

Build a controlled discovery lab with three evidence boundaries:

1 Generate real RSA and ECDSA X.509 artifacts for valid, expired,

future-validity, CA, end-entity, SAN, and no-SAN scenarios.

1 Parse PEM and DER artifacts with a pinned PyCA `cryptography` dependency,

deriving rather than accepting certificate algorithm metadata.

1 Perform a real TLS handshake only against loopback IP literals. Record the

TLS version and cipher suite while leaving negotiated key establishment null unless a future protocol-specific collector can evidence it directly.

Add an evidence-maturity field to every event, inventory record, assessment, and alert. Known collectors have allowed levels. Fixtures are simulation-only; the file parser accepts generated or sanitised captures; the loopback probe is generated evidence only. Authenticated-production and independently-verified claims are represented in the portable schema but rejected at runtime until an authenticated collector trust registry exists.

Because these fields change required contracts, Milestone 4 explicitly advances the certificate event to v1.3, the inventory and assessment contracts to v1.1, and the detection alert to v1.2.

## Consequences

Positive:

- Real cryptographic artifacts now exercise parsing and classification.
- The project can progress without confidential production data.
- Evidence quality and claim boundaries are machine-readable.
- A real local handshake tests certificate-versus-transport separation.
- The probe cannot be repurposed to scan external addresses.

Negative and residual:

- Generated certificates do not reproduce vendor, scale, topology, private-PKI,

HSM, KMS, application-library, or operational edge cases.

- Self-signed lab certificates do not establish trust-chain behavior.
- The standard Python TLS interface used here does not directly expose the

negotiated key-establishment group, so that field remains unknown.

- Private keys are unencrypted because the local TLS server needs them; they

are synthetic test material and must never be reused or imported into a production key store.

- Production coverage and interoperability remain unverified.

## Rejected alternatives

- **Wait for customer data:** rejected because parsers, evidence contracts,

safety checks, and migration workflows are independently testable.

- **Scan public or enterprise targets:** rejected because the milestone has no

target authorisation and must not add an unrestricted network scanner.

- **Infer KEM from TLS cipher or certificate:** rejected because that would

conflate distinct protocol evidence.

- **Label generated artifacts as verified production:** rejected because it

would invalidate the evidence-backed product claim.

# ADR-005: Deterministic migration governance without execution authority

- Status: accepted
- Date: 2026-10-11

## Context

Milestones 1-4 established certificate monitoring, replay-safe evidence, certificate-focused inventory/PQC assessment, and controlled real X.509/TLS discovery. The project goal also includes migration management, but there is no authorised production estate, authoritative ownership source, approved target policy, or change authority.

A credible reference implementation must demonstrate how discovery becomes governed work without presenting fictional data as production coverage or allowing a planning service to perform cryptographic changes.

## Decision

Implement Milestone 5 around six separately validated contracts:

1 Enterprise catalogue with assets, services, owners, suppliers, business/data

context, cryptographic uses, target states, and dependencies.

1 Migration work item with current/target states, priority, wave, blockers,

version, approval identity, and verification criteria.

1 Human decision record bound to an exact work-item version.

2 Time-bounded exception with risk ownership and compensating controls.

3 Independent verification record with pre/post evidence and named checks.

4 Report that keeps discovery coverage and migration progress separate.

Use a fictional six-service enterprise and simulated evidence. Reject duplicate identities, conflicting asset ownership, unknown dependencies, self-dependency, dependency cycles, and evidence-level escalation.

Use function-specific target compatibility:

- key establishment: ML-KEM or a protocol-defined hybrid candidate;
- digital signature and certificate authentication: ML-DSA, SLH-DSA, or a

protocol-defined hybrid candidate.

Target names remain planning candidates. A `supported` value does not prove protocol, product, peer, chain, or fallback interoperability.

Migration priority and wave assignment are deterministic. Missing evidence or required business context remains unassigned. Target readiness and hard dependency failures create explicit blockers.

Require human approval, prevent requester self-approval, reject stale decisions, and keep implementation external. Permit `verified` only after an independent verifier supplies pre-change and post-change evidence and every approved criterion passes. Failed verification becomes `regressed`; inconclusive verification stays open.

## Consequences

Positive:

- The control plane now demonstrates discovery-to-governance flow without real customer data.
- Dependencies, ownership, evidence, targets, decisions, exceptions, and verification are machine-readable and traceable.
- Missing evidence and unsupported targets cannot silently become plans.
- Coverage cannot be used as a substitute for migration completion.
- Planning and verification remain separated from privileged execution.

Negative and residual:

- The catalogue is hand-authored simulation rather than reconciled discovery.
- Priority rules are reference policy, not an organisation-approved risk model.
- Supplier readiness values are fictional and do not establish product support.
- Workflow state is in-memory/file-oriented; no durable multi-user concurrency, identity provider, RBAC service, or tamper-evident audit log exists.
- There is no automated dependency correlation or actual migration execution.

## Rejected alternatives

- **Add a production executor now:** rejected because scope, identity, approval, rollback, and environment authority are not established.
- **Use one overall readiness percentage:** rejected because it would conflate estate visibility with verified migration.
- **Allow algorithm labels to complete work:** rejected because deployed interoperability and downgrade behavior require direct verification.
- **Allow blocked work to be approved normally:** rejected because exceptions need distinct risk ownership, expiry, and compensating controls.
- **Use an LLM to assign authoritative waves:** rejected because security-critical prioritisation must be deterministic, reviewable, and versioned.

# ADR-006: Safe reference foundations for Milestones 6 to 9

- **Status:** accepted
- **Date:** 2026-10-11

## Context

Milestone 5 established non-executing migration governance. The remaining roadmap calls for evidence exchange, a read-only pilot, production control-plane foundations, and optional integrations. This repository has no customer data, written pilot authorisation, production identity provider, operational service, or independently assessed execution environment.

## Decision

Implement all remaining capabilities as a local reference release while preserving the real-world decision gates:

1 Reconcile repeated inventory observations deterministically, retain every

source record, and label contradictory states as conflicts rather than silently selecting truth.

1 Export CBOM-style and dependency views without claiming conformance to an

external CBOM standard.

1 Produce sanitised, Ed25519-signed offline bundles. The signature protects

manifest integrity; signer trust and observation evidence maturity remain separate. Sanitisation and signing never upgrade evidence.

1 Require a versioned, time-bounded, independently approved, read-only scope

before pilot measurement. The included pilot is simulated and cannot satisfy Gate G3.

1 Demonstrate tenant scoping, optimistic versioning, RBAC, separation of

duties, audit, and transactional outbox semantics in local SQLite. This does not satisfy production availability, identity, recovery, or assurance gates.

1 Represent integrations as evidence-bound proposals with no execution

authority. Reject issuance, revocation, rotation, deletion, trust changes, deployment, and generic execution.

## Consequences

Positive:

- the end-to-end governance shape is executable without inventing production

evidence or connecting to customer systems;

- evidence conflicts, source trust, signer trust, and migration completion stay

separate;

- future adapters have explicit least-authority and approval boundaries;
- local tests exercise tenant isolation, stale-write prevention, and

separation-of-duties rules.

Negative and residual:

- SQLite is a reference persistence layer, not the production topology;
- the offline bundle uses an embedded self-signed public key until an external trust registry is approved;
- no real pilot, production SLO, disaster recovery, privacy assessment, penetration test, or external integration has occurred;
- CBOM output is a project-specific reference representation.

## Rejected alternatives

- **Declare all roadmap exit gates complete:** rejected because code and simulated evidence cannot replace authorisation, production operation, or independent assurance.
- **Treat a valid bundle signature as production evidence:** rejected because a signature authenticates bytes only relative to a trusted key; it does not prove the observations are complete or true.
- **Add direct PKI/KMS/HSM execution:** rejected because credential, approval, rollback, blast-radius, and operational controls are absent.

# Threat model: Milestones 1 to 9 reference release

## Assets

- Integrity and provenance of certificate observations
- Organization DNS policy and ownership context
- Alert integrity, severity, evidence, and audit trail
- Collector checkpoints, idempotency records, and preserved raw observations
- Cryptographic inventory records, algorithm evidence, and PQC assessments
- Asset ownership, business criticality, classification, and protection lifetime
- Certificate metadata that may expose internal names or infrastructure
- Future credentials and cryptographic workflow authority (not present today)
- Evidence-maturity labels and the boundary between lab and production claims
- Local TLS target selection and parsed certificate artifacts
- Migration catalogue ownership, dependency, supplier, and target-state data
- Work-item versions, human decisions, exceptions, and verification evidence
- Separation between discovery coverage and verified migration progress

## Trust boundaries




Event, CT fixture, policy, SQLite, and JSONL files can be modified by a local actor. The digest supports comparison but is not a signature or tamper-evident audit record.

## Threats and current mitigations

| Threat | Impact | Current mitigation | Residual / future control |
|---|---|---|---|
| Forged event | False alert or concealed issuance | Strict shape/type/time validation; evidence fields retained | Authenticated collectors and signed provenance |
| Source ID reused with changed content | Evidence substitution | Stored digest comparison fails closed | Source authentication and tamper-evident log |
| Duplicate/replayed observation | Alert flooding | Durable source identity/digest idempotency | Distributed idempotency key and retention policy |
| Checkpoint loss or cycle | Missed data or infinite collection | SQLite checkpoints, page-completion updates, cursor-cycle and page limits | Replicated state, monitoring, recovery procedure |
| Policy tampering | Suppressed or flooded alerts | Policy is explicit and fail-closed on invalid data | Signed/versioned policy, change approval, independent ownership source |
| DNS normalization bypass | Unknown name treated as trusted | Lowercase, trailing-dot removal, IDNA normalization, wildcard restrictions, label-boundary suffix matching | Differential tests with production parser and public-suffix awareness |
| Replay/duplicate event | Alert flooding | Deterministic alert identifier | Durable deduplication store and replay window |
| Prompt/model manipulation | Misleading response or unsafe action | No model call; explanation cannot change detection/severity; no executor | Structured output validation, evaluation, isolation, human review |
| Sensitive-name leakage | Infrastructure disclosure | Local-only sink and minimal fields | Classification, redaction, encryption, tenant isolation, retention policy |
| Malicious JSON volume | Resource exhaustion | Bounded local CLI use only | Payload/field limits, streaming parser, quotas, backpressure |
| JSONL injection | Corrupted downstream parsing | One encoded JSON object per line | Authenticated structured transport and schema enforcement |
| Privilege creep | Unauthorized issuance/revocation/key action | No credentials, network calls, or action interfaces | Separate executor, least privilege, approvals, audit, rollback |
| Algorithm-label spoofing | False PQC-readiness claim | Closed algorithm family enum, OID shape validation, provenance retained | Parse signed source artifacts; independently verify implementations and chains |
| Missing coverage presented as safety | Unmanaged vulnerable cryptography remains hidden | Explicit `certificate-only` coverage and limitations; unknown stays unknown | Coverage metrics across network, code, cloud, PKI, KMS, and HSM collectors |
| Business-context poisoning | Incorrect migration order | Evidence fields remain separate from deterministic classification | Authoritative CMDB/owner sources, change approval, conflict handling |
| PQC or hybrid label treated as deployment proof | Premature completion claim | Status is `candidate-quantum-resistant` or `hybrid-unverified`, never safe/compliant | Protocol interoperability, chain, implementation, and relying-party verification |
| Sensitive inventory disclosure | Attacker gains infrastructure and dependency intelligence | Local-only reference storage | Encryption, tenant isolation, field-level authorization, audit, retention controls |
| Fixture promoted to production evidence | Test result presented as customer assurance | Known collectors have allowed evidence levels; fixture sources are simulation-only | Authenticated collector identity and signed evidence envelopes |
| Local TLS probe used for network scanning | Unauthorised access or SSRF-style reachability | IP literals only; loopback addresses only; bounded timeout; no redirect or credential support | Separate reviewed production collector with explicit scope and egress policy |
| Certificate parser resource abuse | Memory/CPU exhaustion or parser exploitation | One-megabyte input bound, pinned parser dependency, malformed input fails closed | Sandboxed parser service, quotas, dependency monitoring |
| TLS cipher label used as KEM proof | False conclusion about key establishment | Negotiated key establishment is explicitly `null` unless directly evidenced | Protocol-specific telemetry and independent handshake verification |
| Synthetic private key leakage | Lab key reused or mistaken for production material | Factory returns ephemeral, unencrypted keys only for local tests; artifacts labelled generated | Ephemeral storage, restrictive file mode, automated cleanup, never import into production stores |
| Fictional catalogue presented as enterprise truth | False assurance and wrong migration plan | Catalogue and report are `simulated`; high-assurance claims rejected | Authorised reconciliation against authenticated asset sources |
| Dependency graph poisoning or cycle | Unsafe ordering or planning deadlock | Unknown/self/duplicate dependencies and cycles fail closed | Authenticated sources, conflict lineage, graph-owner review |

| Threat | Impact | Current mitigation | Residual / future control |
|---|---|---|---|
| Conflicting ownership | Work routed to the wrong authority | Conflicting asset ownership fails closed | Authoritative ownership source and dispute workflow |
| Functionally wrong target | KEM used as signature target or signature scheme used for key establishment | Function-specific target allowlists; unsupported target blocks work | Versioned organisation policy and protocol-specific profiles |
| Self-approval or stale approval | Unauthorised or outdated migration decision | Requester cannot approve; decision binds exact object version | Identity provider, RBAC, dual control, signed audit event |
| Permanent or expired exception | Risk remains unmanaged | Separate exception contract, expiry, risk owner, compensating controls | Scheduled expiry enforcement and escalation |
| Weak verification represented as completion | False migration completion | Independent verifier, exact target match, all criteria pass, pre/post evidence required | Authenticated evidence sources and independent assurance sampling |
| Coverage conflated with completion | Misleading programme reporting | Separate report sections and percentages; no overall score | Portfolio governance review and metric definitions |

## Security invariants

1 Invalid events or policy fail closed and produce no alert output.

2 Explanations cannot create alerts or change rule results, severity, or score.

3 DNS suffix matching occurs only at label boundaries.

4 Alert identifiers are stable for the same event and rule.

5 The pipeline performs no network or cryptographic lifecycle side effects.

6 The same collector/source identity cannot be accepted with a different raw

payload digest.

1 Checkpoints advance only after all entries on a validated page are recorded.

2 Missing algorithm evidence is `unknown`, never quantum-resistant by default.

3 PQC and hybrid observations require deployment verification and are not

labelled compliant or safe solely from algorithm names.

1 Certificate evidence does not prove negotiated key establishment or complete

enterprise cryptographic coverage.

1 Observation, inventory record, and PQC assessment are committed atomically.

2 Simulation and generated evidence cannot claim authenticated-production or

independently-verified status; both high-assurance levels are runtime- disabled until an authenticated collector trust registry exists.

1 The lab TLS probe cannot connect to a non-loopback target.

2 Cipher-suite and certificate observations do not invent negotiated

key-establishment evidence.

1 Catalogue evidence cannot claim a higher maturity than its source boundary.

2 Dependency cycles, unknown dependencies, duplicate identities, and

conflicting asset ownership fail closed.

1 A requester cannot approve their own migration or exception.

2 Decisions, exceptions, and verification records bind to the exact current

work-item version; stale records fail closed.

1. Exceptions are time bounded and require compensating controls.
2. `verified` requires independent pre/post evidence and every approved

criterion to pass.

1. Discovery coverage and migration completion remain separate measures.
2. The migration workflow performs no production change.

# Milestones 6 to 9 threat-model delta

## New assets

- reconciliation lineage and conflict state;
- offline bundle manifest, signature, public key, and sanitised artifacts;
- pilot authorisation, scope denominator, expiry, and stop conditions;
- tenant identifiers, workflow versions, audit events, and outbox records;
- integration proposals and evidence references.

## New threats and controls

| Threat | Required control |
|---|---|
| Conflict laundering | Retain every source record and label contradictory states; canonical selection is not truth resolution |
| Sanitisation interpreted as stronger evidence | Preserve the weakest enclosed evidence level and report sanitisation separately |
| Self-signed bundle treated as authenticated source | Verify integrity but report signer trust separately; no automatic evidence upgrade |
| Bundle artifact tampering | Verify exact manifest names, canonical sizes, SHA-256 digests, and Ed25519 signature |
| Pilot scope creep or expired authority | Read-only authorisation, independent approver, explicit assets, validity window, and stop conditions |
| Cross-tenant object access | Tenant included in every workflow primary key and lookup |
| Lost update or replayed approval | Expected-version transition and audit/outbox event in one transaction |
| Self-approval or unauthorised role | Role gate plus requester/approver separation |
| Proposal mistaken for execution authority | Schema fixes status to approval-required and execution authority to none |
| Direct PKI/KMS/HSM mutation | Reject issue, revoke, rotate, delete, deploy, trust-change, and generic execute operations |

## Residual risk

The reference controls do not establish production identity assurance, database row-level security, high availability, disaster recovery, secure key custody, privacy compliance, connector credential safety, or external penetration-test assurance. Those remain Gate G3-G5 evidence requirements.

# Out of scope

Production authentication/authorization, production multi-tenancy and service availability, CA validation, CT wire parsing/proof verification, source authentication, distributed event delivery, production SIEM authentication, authoritative CMDB enrichment, Internet or enterprise-network scanning, authenticated production discovery, non-certificate discovery, durable multi-user operations, production approval identity, connector credentials, and remediation/orchestration.

# SOC playbook: CERT-001 - certificate for an unknown DNS namespace

## Purpose

Investigate a certificate issuance event containing at least one DNS name that does not match the approved organization namespace policy. A detection is not proof of malicious issuance or domain ownership.

## Initial triage

1 Confirm the alert evidence, event identifier, source, observation time,

certificate fingerprint, issuer, subject, and unmatched DNS names.

1 Validate the source independently. For public certificates, inspect the

relevant Certificate Transparency evidence and certificate chain; do not rely solely on the alert explanation.

1 Check authoritative domain/asset ownership records, recent acquisitions,

approved third-party services, and pending DNS or certificate requests.

1 Search for related certificates, DNS changes, TLS endpoints, account

activity, and issuance events using the fingerprint, serial number, issuer, subject, and DNS names.

1 Determine whether the policy is stale, the event is malformed, the name is a

legitimate external dependency, or the issuance is unauthorized.

## Escalation

- Escalate immediately for confirmed organization names issued by an unexpected

CA, active impersonation, exposed services, compromised accounts, or evidence of unauthorized control.

- Engage PKI, DNS, cloud, identity, legal/brand, and incident-response owners as

applicable.

## Containment and recovery

Do not revoke, replace, block, or modify DNS solely because this rule fired. Use established incident and change procedures. If unauthorized issuance is confirmed, coordinate CA/account containment, revocation where applicable, credential rotation, DNS/service remediation, monitoring, and evidence preservation with the responsible owners.

## Closure evidence

Record the ownership decision, authoritative sources checked, policy correction if any, related indicators, actions/approvals, residual risk, and monitoring or follow-up requirements.